

Fixing the Fixpoint: A Formal Theory of Convergence Detection for Incremental Recursive Computation

ANONYMOUS AUTHOR(S)

Modern incremental computation theories like DBSP have enabled efficient incrementalization of general recursive computations. To do so, they require a runtime Fixpoint Detection (FPD) mechanism to detect whether an iterative computation has reached the fixpoint and thus should terminate. However, we show that the commonly suggested “FirstZero” strategy is unsound even in naturally arising cases, and that exact FPD is impossible for arbitrary DBSP circuits with expressive primitive nodes. This issue reveals a fundamental gap between the mathematical specification and implementations of such theories. To fill this gap, using DBSP as a core calculus, we develop a formal theory of convergence detection. Within this theory, we define *internal convergence* (IntConv) as a declarative criterion corresponding to the internal-state-stability strategy used by practical implementations, and prove that IntConv is a sufficient condition for external convergence. We then define the state fixpoint (StFP) predicate and a sound and complete StFP detector. Combining the StFP detector with fixed-input and zero-output checks yields a sound and complete IntConv detector. Moreover, for a large class of useful circuits (programs) including Datalog queries, nested while queries, and their incrementally optimized versions, we show that IntConv is not only sound but also complete (meaning any convergence in the theory implies the convergence in our criterion). As a result, our theory provides semantic guarantees for convergence detection on all these circuits. Our results are formally verified in Lean.

CCS Concepts: • **Software and its engineering** → **Incremental compilers**; • **Theory of computation** → **Streaming models**; **Database query processing and optimization (theory)**.

Additional Key Words and Phrases: fixpoint detection, stream-based computing, incremental recursive computation

ACM Reference Format:

Anonymous Author(s). 2018. Fixing the Fixpoint: A Formal Theory of Convergence Detection for Incremental Recursive Computation. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 27 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

Incremental computation [23, 29] is a cornerstone of efficient software systems. Its fundamental premise is straightforward: when the input to a computation changes, the system should update the output by processing only the small changes (or “deltas”), rather than recomputing the entire result from scratch. This principle finds its most classical application in databases in the form of *Incremental View Maintenance* (IVM) [10, 20], as shown in Fig. 1(a). For standard relational queries, IVM is well-understood and widely deployed, allowing systems to maintain up-to-date views with significantly reduced cost compared to recomputation.

However, modern data analysis has evolved beyond simple relational queries. Applications ranging from graph analytics to machine learning rely heavily on algorithms that are *iterative or*

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference acronym 'XX, Woodstock, NY

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/2018/06

<https://doi.org/XXXXXXX.XXXXXXX>

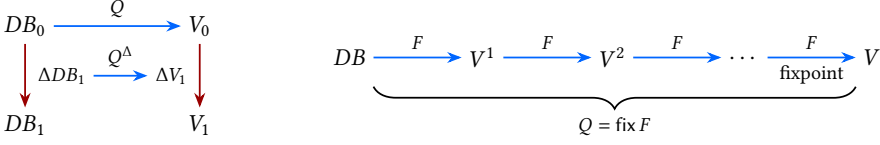


Fig. 1. (a) Incremental view maintenance; (b) iterative fixpoint computation for a recursive query.

recursive in nature. As a simple example, consider the simple recursive query Q for computing the transitive closure of a finite directed graph, expressed in Datalog as follows:

```
// Edge relation with head and tail
input relation E(h: Node, t: Node)
// Reach relation with source s and sink t
output relation R(s: Node, t: Node)
R(x, y) :- E(x, y).
R(x, y) :- E(x, z), R(z, y).
```

The semantics of this recursive query is defined in terms of the non-recursive query $F_E(R) := E \cup (E \circ R)$. The result of the recursive query, Q , is defined as the least fixpoint of the equation $x = F_E(x)$. In this case, the process of finding the least fixpoint can be understood in terms of shortest distances: if R^n consists of pairs of nodes for which the length of the shortest connecting path is $\leq n$, then $F_E(R^n) = R^{n+1}$ consists of pairs of nodes whose shortest distance is $\leq n + 1$; consequently, the least fixpoint is the transitive closure of the graph. This intuition corresponds to the following iterative computation strategy: repeatedly apply F_E to the accumulated result until a fixpoint is reached, as shown in Fig. 1(b) (often called *Kleene iteration*).

It is challenging, however, to combine incremental computation with recursion to *efficiently incrementalize recursive queries*—that is, in our example, to update the graph and efficiently recompute the new transitive closure. Luckily, frameworks such as DBSP [6] and Differential Dataflow [26] give elegant theories for efficient incrementalization of general recursive queries. However, we find that there is a fundamental gap between their mathematical specification and implementations of such frameworks, having to do with detecting fixpoints. The *Fixpoint Detection* (FPD) problem actually involves two issues:

- (1) If the choice of primitive nodes makes DBSP Turing-complete, exact FPD for arbitrary circuits is *impossible*: there is a fixed level-1 circuit with no computable detector that is both sound and complete (§4.2).
- (2) In the DBSP paper [6], the authors suggest that for many practical queries, including Datalog queries, a key primitive for fixpoint detection can be implemented by an iterative procedure that can be stopped when the first “0” is encountered (e.g., the first $\emptyset$ when working with sets), which intuitively implies that there is no change in two adjacent results. We call this the *FirstZero* strategy. However, in §2.1 (and formalized in §4.1), we show that the FirstZero strategy is unsound, including in naturally arising cases; that is, using it in the implementation results in output that does not match the mathematical specification.

These issues are not confined to DBSP; as discussed in §9, similar problems exist in other frameworks supporting incrementalization of general recursive queries, such as Differential Dataflow [26].

To fill in this gap, we develop a formal theory of convergence detection using DBSP as a core calculus. Within this theory, we define *internal convergence* (IntConv) as a declarative convergence criterion corresponding to the internal-state-stability strategy used by practical implementations such as Feldera [14]. We show that IntConv is a *sound* criterion, so it never produces an incorrect

answer (§5).¹ At the end of §5, we give a theoretical data-dependency example in which relying only on external convergence can require recursive trace-backs and additional retained state. This example motivates an implementation benefit of IntConv. To connect this declarative criterion to runtime state, we define the State Fixpoint (StFP) predicate and provide a sound and complete StFP detector (§6). StFP and the StFP detector give semantic and algorithmic formalizations, respectively, of the internal-state-stability strategy in our denotational model. The global IntConv detector combines the StFP detector with fixed-input and zero-output checks across all bracketed sub-circuits, yielding sound and complete IntConv detection. Because the predicate and StFP detector are structural, our formulation applies compositionally to nested recursive computations; §9 compares this capability with the Feldera implementation [14]. Moreover, we show that IntConv is *complete*—i.e., it detects convergence whenever the theoretical specification does—for a large class of useful circuits (§7). In particular, we use *regular circuits* as a name for the standard DBSP recursive sublanguage generated by lifted scalar circuits, composition, Datalog queries, and while queries. We then show that IntConv is complete for regular circuits and their incrementally optimized versions.² As a result, sound and complete convergence detection is possible for all these useful circuits.

Our key results are formally verified in Lean 4 [11]. Our Lean 4 development is described in §8; all mechanized proofs are available as anonymous supplementary material.

In summary, our contributions are as follows:

- We identify and formalize the Fixpoint Detection (FPD) problem in DBSP, showing that the commonly used FirstZero criterion is unsound in general and that exact FPD is impossible for arbitrary expressive circuits.
- We define internal convergence (IntConv) as a declarative criterion corresponding to the internal-state-stability strategy used by Feldera-style implementations, and prove that it is a sound and sufficient condition for convergence. We define the State Fixpoint (StFP) predicate and a sound and complete StFP detector, giving semantic and algorithmic formalizations of this strategy. Combining the StFP detector with fixed-input and zero-output checks across all bracketed sub-circuits yields a sound, complete, and compositional global IntConv detector.
- We show that IntConv is a complete criterion for a large class of useful circuits, including all regular circuits and their incrementally optimized forms. As a result, exact convergence detection is possible for these circuits despite its impossibility for the full language.
- We formally verify our key results in Lean.

2 Overview

2.1 DBSP and the Unsoundness of the FirstZero Strategy

To lay the groundwork for the problems that we address, we first discuss how DBSP implements and incrementalizes the Datalog query for transitive closure (§1) in three steps. DBSP uses *circuits* to express computations. It first compiles the query into a “naive” circuit, then optimizes it into a “semi-naive” circuit, and finally optimizes it further into a “delta-of-deltas” circuit. What matters for our purposes is what is computed incrementally in these circuits; Figure 2 depicts the intermediate results of the evaluation for one graph and one update to it. In the figure, the vertical dimension represents changes to the input, i.e., the *outer* iterations for incremental computation (e.g., we

¹Because exact FPD for arbitrary circuits is impossible, IntConv is not complete for general circuits. Consequently, there can be computations that mathematically converge, but convergence is not detected by the IntConv criterion.

²I.e., if a computation of such a circuit mathematically converges, convergence will always be detected by the IntConv criterion.

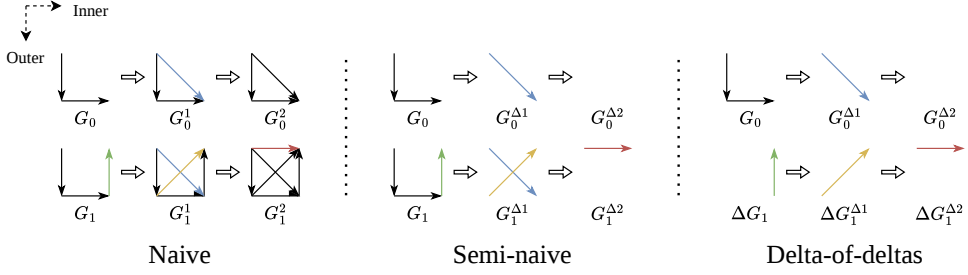


Fig. 2. Intermediate-evaluation results arising in the three DBSP strategies for incrementalizing a transitive-closure query.

add a green edge to the first input G_0 to get the second input G_1), while the horizontal dimension represents the *inner* iterations for fixpoint computation (e.g., we apply the single-step query F to G_0 to get G_0^1). The goal after each outer iteration m is to obtain the transitive-closure “view” for graph G_m .

The **naive** strategy is not optimized for incremental computation at all: for each outer iteration, it re-runs the entire fixpoint computation from scratch; on each inner iteration, it merely applies the non-recursive query F to intermediate results (until the inner-iteration fixpoint is obtained). Its intermediate result in the n^{th} inner iteration of the m^{th} outer iteration G_m^n is the set of pairs of nodes whose distance is within $n + 1$ in G_m .

The **semi-naive** strategy optimizes the naive strategy along the horizontal dimension (inner iterations). The name comes from the Datalog semi-naive evaluation strategy (Algorithm 2 from the book [18]). It computes inner-iteration *deltas* of the intermediate results of the naive strategy (e.g., $G_0^{\Delta 1} = G_0^1 - G_0$ contains only the blue edge). The intermediate result $G_m^{\Delta n}$, for $n \geq 1$, consists of pairs of nodes whose distance is exactly $n + 1$ in G_m . In each outer iteration, when the inner-fixpoint is reached, the intermediate results can be collected—e.g., $G_1 \cup G_1^{\Delta 1} \cup G_1^{\Delta 2}$ —to obtain the final result.

The **delta-of-deltas** strategy further incrementalizes the semi-naive strategy along the outer iterations by computing *outer-iteration deltas of the inner-iteration deltas* (e.g., $\Delta G_1^{\Delta 1} = G_1^{\Delta 1} - G_0^{\Delta 1}$ contains only the yellow edge). For each outer iteration, it accepts the delta of the input graph (e.g., ΔG_1), and upon obtaining the inner fixpoint, the inner-iteration intermediate results can be collected to obtain the delta of the transitive-closure view. For instance, for outer iteration 1, the delta of the view is the three pairs in $\Delta G_1 \cup \Delta G_1^{\Delta 1} \cup \Delta G_1^{\Delta 2}$. Note that this value equals $G_1^2 - G_0^2$.

To produce correct answers, all three strategies need a common component we call *Fixpoint Detection* (FPD): a runtime mechanism to detect whether an inner-iteration computation has already reached a fixpoint, at which point, it can terminate the current inner iteration, collect the intermediate results (for the semi-naive and delta-of-deltas strategies), and proceed to the next outer iteration.

In the DBSP theory, FPD and inner-iteration collection is encapsulated in a mathematical operator called *stream elimination*, denoted by f , which takes an infinite stream $s \in (\mathcal{S}_A := \mathbb{N} \rightarrow A)$ representing the intermediate results of an *unbounded* iterative computation, and produces a scalar value $a \in A$ representing the integrated final result. Intuitively, $f(s)$ is defined to return the sum of all *non-zero* elements in s . f can only be defined as a *partial* function: it only produces an answer for streams that have a *finite* number of non-zero elements (in which case we say that $f(s)$ converges). When $f(s)$ diverges, the implementation can simply run forever or terminate after a timeout. Either is semantically consistent: DBSP is designed to be Turing-complete [6] due to unbounded loops. The question that really matters is “how can the system detect when a stream s is all zeros beyond

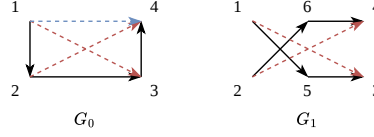


Fig. 3. A pair of input graphs that demonstrate that the FirstZero criterion applied to the delta-of-deltas strategy computes an incorrect answer. Instead of the correct transitive closure of G_1 , namely, $\{(1, 5), (2, 6), (5, 3), (6, 4), (1, 3), (2, 4)\}$, we would obtain $\{(1, 5), (2, 6), (5, 3), (6, 4), (1, 3), (2, 4), (1, 4)\}$.

some point?” in which case $f(s)$ indeed converges, and the semantics demands that the answer be produced. The challenge is that the implementation has only a finite prefix of s at run time, and it must concretely decide all subsequent values of s are zero, but there are infinitely many of them. Finding a way to establish s has finitely many non-zero values based on a finite prefix is the essence of the **FPD problem** in DBSP.

In the DBSP paper [6], the authors suggest that for many practical queries including the Datalog queries, f can be “approximated without loss of precision by integrating until it encounters the first 0”. We call this criterion for deciding convergence the FirstZero criterion (or FirstZero strategy).

For the naive and the semi-naive strategies,³ the FirstZero criterion is indeed a sound criterion, because the first zero in the delta stream ($G_m^{\Delta n} = 0$) implies that the fixpoint of the original stream has been reached ($G_m^n = F(G_m^{n-1}) = G_m^{n-1}$), so $G_m^{\Delta p}$, for $p \geq n$ must all be zero.

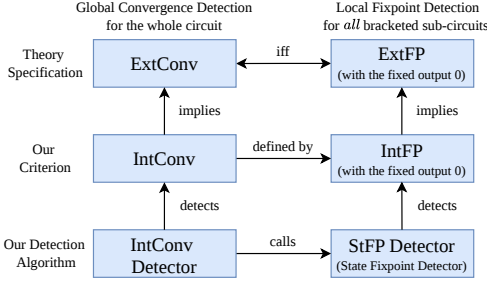
However, for the delta-of-deltas strategy, the FirstZero criterion is, in general, incorrect. That is, if a runtime executes the stream elimination operator f in a DBSP operator by summing until the first zero, it produces an incorrect result on some inputs where there are more non-zero elements that should be summed; there is a convergence point in the input, but it is further ahead in the stream than the first zero.

For a concrete example where FirstZero produces an incorrect answer, consider the two graphs G_0 and G_1 shown in Fig. 3. In DBSP, the delta between two databases can have negative quantities, representing deletions. For G_0 and G_1 , the delta ΔG_1 consists of negative edges $\{(1, 2), (2, 3), (3, 4)\}$ and positive edges $\{(1, 5), (2, 6), (5, 3), (6, 4)\}$. Note that after the first step of the semi-naive strategy, $G_0^{\Delta 1}$ and $G_1^{\Delta 1}$ are exactly the same (the positive red edges $\{(1, 3), (2, 4)\}$). Consequently, after the first step of the delta-of-deltas strategy, $G_0^{\Delta 1}$ consists of $\{(1, 3), (2, 4)\}$ and $\Delta G_1^{\Delta 1} = G_1^{\Delta 1} - G_0^{\Delta 1} = \emptyset$ (because subtraction operates over $\mathbb{Z}$ -sets, elements with identical positive counts cancel out)—i.e., it is the first zero, so the FirstZero criterion would say, incorrectly, that the inner-iteration fixpoint for ΔG_1 has been reached.

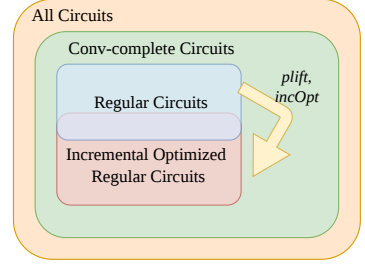
When the delta-of-deltas strategy is properly performed, $\Delta G_1^{\Delta 1}$ is $\emptyset$, but the subsequent $\Delta G_1^{\Delta 2}$ is not $\emptyset$: $G_0^{\Delta 2}$ consists of the blue edge $\{(1, 4)\}$, and thus $\Delta G_1^{\Delta 2}$ consists of the *negative* blue edge $\{(1, 4)\}$. Only when we get to $G_0^{\Delta 3}$ and $\Delta G_1^{\Delta 3}$ do we have $\emptyset$ at corresponding inner-iteration spots of both outer iterations. The collected inner-iteration intermediate results are $\Delta G_1 \cup \Delta G_1^{\Delta 1} \cup \Delta G_1^{\Delta 2} \cup \Delta G_1^{\Delta 3}$, which has negative edges $\{(1, 2), (2, 3), (3, 4), (1, 4)\}$ and positive edges $\{(1, 5), (2, 6), (5, 3), (6, 4)\}$. When combined with the transitive closure of G_0 , i.e., positive edges $\{(1, 2), (2, 3), (3, 4), (1, 3), (2, 4), (1, 4)\}$, we obtain the transitive closure of G_1 , $\{(1, 5), (2, 6), (5, 3), (6, 4), (1, 3), (2, 4)\}$.

In contrast, the FirstZero criterion produces an incorrect result. The collected inner-iteration intermediate results are $\Delta G_1 \cup \Delta G_1^{\Delta 1}$, which has negative edges $\{(1, 2), (2, 3), (3, 4)\}$ and positive edges $\{(1, 5), (2, 6), (5, 3), (6, 4)\}$. When combined with the transitive closure of

³In the DBSP circuit for the naive strategy, the intermediate results are in fact first differentiated to become deltas and then sent to f , so the input to f is the same in the circuits for the naive and semi-naive strategies.



(a) Overview of the IntConv theory.



(b) Overview of the Conv-complete circuits.

Fig. 4. Overview of the concepts that play a role in our theory.

G_0 , i.e., positive edges $\{(1, 2), (2, 3), (3, 4), (1, 3), (2, 4), (1, 4)\}$, we obtain the incorrect result $\{(1, 5), (2, 6), (5, 3), (6, 4), (1, 3), (2, 4), (1, 4)\}$, which has the extra positive edge $(1, 4)$.

2.2 Overview of Our Theory

The counter example to the soundness of FirstZero as a convergence criterion in §2.1, together with the impossibility result in §4.2, reveals a gap between the mathematical theory of DBSP and its practical implementation. Our theory addresses this gap through a simple progression: specify convergence mathematically, strengthen that specification to require internal stability, and turn internal stability into a runtime check. This progression operates at two scales—locally for one iterative computation and globally for a complete circuit.

To express these two scales uniformly, we define a core calculus of well-formed DBSP circuits (§3.2). In this calculus, δ_0 initiates an unbounded iterative computation and f integrates its result; we call the circuit between them a *bracketed circuit*. The Fixpoint Detection (FPD) problem asks when one such bracketed circuit reaches a fixpoint. A complete circuit may contain multiple, potentially nested bracketed circuits, so the global *Convergence Detection* problem asks when every bracketed sub-circuit reaches a fixpoint. We formalize both problems and show that exact FPD is impossible for arbitrary circuits with expressive primitive nodes (§4). The result does not apply to Datalog or regular circuits, for which our later completeness theorems yield exact detection.

Fig. 4a organizes the local and global problems into three layers. At the specification layer, *external fixpoint* (ExtFP) says that a circuit's input and output are fixed after a specified iteration; when the fixed output is 0, ExtFP is equivalent to the original FPD specification. Its global counterpart, *external convergence* (ExtConv), requires every bracketed sub-circuit to converge. At the criterion layer, inspired by the internal-state-stability criterion used in Feldera [14], *internal fixpoint* (IntFP) strengthens ExtFP by requiring every internal node to reach an external fixpoint, and *internal convergence* (IntConv) requires the inner circuit of every bracketed sub-circuit to satisfy IntFP with fixed output 0. Consequently, IntFP implies ExtFP and IntConv implies ExtConv (§5). At the detection layer, *state fixpoint* (StFP) connects the internal criterion to the state-oriented check performed at runtime.

The distinction between external and internal convergence also provides theoretical evidence for implementation benefits of IntConv. At the end of §5, we give a data-dependency example in which an evaluation that relies only on ExtConv can require recursive trace-backs and additional retained state. This example is a qualitative example, not an empirical performance evaluation or a general lower bound.

We show that IntConv is indeed detectable using StFP and a high-level detector (§6). Although our denotational semantics treats circuits as pure functions, implementations maintain state; StFP asks whether the circuit would be at an internal fixpoint if its observed input remained fixed. We give a structural StFP detector that is sound and complete for StFP. Together with a fixed-input check, it yields a local IntFP detector. The sound and complete global IntConv detector applies this local detector to every bracketed sub-circuit and checks that the fixed output is 0. Because the StFP detector follows circuit structure, it also applies uniformly to nested bracketed circuits. We will compare this capability with the Feldera implementation in §9.

Finally, we identify useful classes of circuits on which the internal and external specifications coincide, as shown in Fig. 4b. We call a circuit *convergence-complete* (Conv-complete) when its ExtConv implies IntConv for every input. On these circuits, the implementable internal criterion exactly matches the mathematical external convergence specification. We use *regular circuits* as a name for the standard DBSP recursive sublanguage generated by lifted scalar circuits, sequential and parallel composition, Datalog queries, and while queries, including their nested uses. We prove that regular circuits are Conv-complete. We also formalize DBSP's incremental optimization as the syntax-directed transformations plift and incOpt and prove that both preserve Conv-completeness, yielding the classes summarized in Fig. 4b (§7).

3 Preliminary: A Formal Language for DBSP

In this section, we first introduce the core concepts of DBSP that are necessary to understand the FPD problem and our solution (§3.1), then present our formal language for DBSP well-formed circuits (§3.2), and finally show how the Datalog query from §1 is implemented and incrementalized as an example (§3.3).

3.1 Streams and Operators in DBSP

In this section, we introduce the core DBSP theory, which forms the universe for our language's semantics.

Definition 3.1 (Stream). Given an Abelian group $(A, +, 0_A, -)$, a *stream* of values from A , or an A -stream, is a function $s : \mathbb{N} \rightarrow A$. We denote the set of all A -streams by $\mathcal{S}_A = \{s \mid s : \mathbb{N} \rightarrow A\}$. We write $s[t]$ for the t -th element of stream s .

Streams inherit the group structure pointwise: $(s_1 + s_2)[t] = s_1[t] + s_2[t]$, so $(\mathcal{S}_A, +, 0_{\mathcal{S}_A}, -)$ is itself an Abelian group.

Definition 3.2 (Stream Operator). A *stream operator* is a function $T : \mathcal{S}_{A_1} \times \cdots \times \mathcal{S}_{A_n} \rightarrow \mathcal{S}_B$.

In circuit diagrams, we depict stream operators as blocks and streams as arrows.

Definition 3.3 (Lifting). Given a function $f : A \rightarrow B$, its *lifting* $\uparrow f : \mathcal{S}_A \rightarrow \mathcal{S}_B$ applies f pointwise in time: $(\uparrow f)(s)[t] = f(s[t])$. Lifting distributes over composition: $\uparrow(f \circ g) = (\uparrow f) \circ (\uparrow g)$.

Definition 3.4 (Delay). The *delay operator* $z^{-1} : \mathcal{S}_A \rightarrow \mathcal{S}_A$ shifts a stream by one time step, inserting zero at $t = 0$: $z^{-1}(s)[t] = 0_A$ if $t = 0$, and $s[t - 1]$ for $t \geq 1$.

Definition 3.5 (Feedback Cycle). For certain operators $T : (\mathcal{S}_A \times \mathcal{S}_B) \rightarrow \mathcal{S}_B$ and $F : \mathcal{S}_B \rightarrow \mathcal{S}_B$, the equation $Q(s) = T(s, F(Q(s)))$ has a unique solution $Q : \mathcal{S}_A \rightarrow \mathcal{S}_B$, denoted as $\lambda s. \text{fix } \alpha. T(s, F(\alpha))$.⁴ Feedback cycles can be represented as Fig. 5a.

DBSP uses feedback cycles mainly for two purposes: express iterative computation and construct integration operator I , as we will show in the following.

Definition 3.6 (Integration and Differentiation). The *integration operator* $I : \mathcal{S}_A \rightarrow \mathcal{S}_A$ is $I(s) = \text{fix } \alpha. (s + z^{-1}(\alpha))$, computing the running sum: $I(s)[t] = \sum_{i \leq t} s[i]$, as shown in Fig. 5b. The *differentiation operator* $\mathcal{D} : \mathcal{S}_A \rightarrow \mathcal{S}_A$ is $\mathcal{D}(s) = s - z^{-1}(s)$, as shown in Fig. 5c.

⁴Our formal language guarantees that all feedback cycles are well-defined.

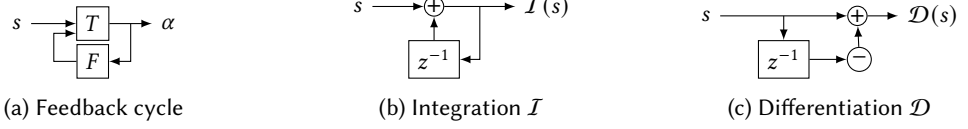


Fig. 5. Some core DBSP operators represented as circuit diagrams.

I and $\mathcal{D}$ are inverses ($I \circ \mathcal{D} = \mathcal{D} \circ I = \text{id}$). Given a stream operator $Q : \mathcal{S}_A \rightarrow \mathcal{S}_B$, its *incremental version* is defined as $Q^\Delta = \mathcal{D} \circ Q \circ I$. The operator Q^Δ consumes a stream of changes and produces a stream of changes: if $Q(s_1) = s_2$, then $Q^\Delta(\mathcal{D}(s_1)) = \mathcal{D}(s_2)$. The DBSP theory establishes compositional properties for Q^Δ , such as the chain rule $(Q_1 \circ Q_2)^\Delta = Q_1^\Delta \circ Q_2^\Delta$. This leads to a syntax-guided incrementalization algorithm, which we will formalize as `incOpt` in §3.2.

To incrementalize iterative computations (like recursive Datalog queries), we need to introduce a new “inner” time dimension for iterative fixpoint computation besides the “outer” time dimension for incremental computation, as §2.1 shows. DBSP implements this via two specific operators and nested streams.

Definition 3.7 (Stream Introduction). The *stream introduction* operator $\delta_0 : A \rightarrow \mathcal{S}_A$ produces a stream from a scalar value v : $\delta_0(v)[0] = v$ and $\delta_0(v)[t] = 0$ for $t > 0$.

Definition 3.8 (Stream Elimination). The *stream elimination* operator $f : \mathcal{S}_A \rightarrow A$ aggregates a stream that is *zero almost everywhere* (i.e., $\exists t_0 \in \mathbb{N}. \forall t \geq t_0. s[t] = 0_A$) into a scalar value: $f(s) = \sum_{t \geq 0} s[t]$.

As we mentioned in §2.2, δ_0 takes a scalar input and outputs a stream and initiates a new unbounded iterative computation. f integrates the intermediate results of the inner computation back into a single scalar output. If the input stream of f is not zero almost everywhere, the summation diverges. This potential for divergence leads to the FPD problem that we address in this paper.

A *nested stream* (or stream of streams) has type $\mathcal{S}_{\mathcal{S}_A} = \mathbb{N} \rightarrow (\mathbb{N} \rightarrow A)$. It can be viewed as an infinite 2D matrix where (t_0, t_1) index the outer time and inner time, respectively, and each row is an inner stream.

We call an operator $S : \mathcal{S}_{\mathcal{S}_A} \rightarrow \mathcal{S}_{\mathcal{S}_B}$ a *nested operator*. Just as we lift scalar functions to operators, we can lift operators to nested operators. If $S : \mathcal{S}_A \rightarrow \mathcal{S}_B$, then $\uparrow S : \mathcal{S}_{\mathcal{S}_A} \rightarrow \mathcal{S}_{\mathcal{S}_B}$ applies S to each *row* of the nested stream: $(\uparrow S)(s)[t_0] = S(s[t_0])$. For a scalar function $f : A \rightarrow B$, we lift it twice to get $\uparrow\uparrow f : \mathcal{S}_{\mathcal{S}_A} \rightarrow \mathcal{S}_{\mathcal{S}_B}$, defined as $(\uparrow\uparrow f)(s)[m][n] = f(s[m][n])$.

For nested streams, delay z^{-1} shifts values along the outer time dimension, delaying “rows” of the matrix, while lifted delay $\uparrow z^{-1}$ shifts values along the *inner* time dimension, delaying “columns” of the matrix. The integration I operates along the outer time dimension, summing up rows of the matrix, while the lifted integration $\uparrow I$ operates along the inner time dimension, summing up columns of the matrix.

3.2 A Formal Language for DBSP Well-Formed Circuits

We now formalize a language of DBSP *well-formed circuits* (which we simply abbreviate as *circuits* from now on) that serves as the core calculus for our theory. This circuit language is an inductive grammar as shown in Table 1.

A circuit $c : A \rightarrow B$ transforms an input stream of type A to an output stream of type B . Types are built from *base types* (Abelian groups) and binary products $A \times B$. Each circuit is annotated with

a *stream level* $\ell \in \{1, 2\}$,⁵ representing whether the circuit's input and output are streams (level 1) or nested streams (level 2). For instance, a bracket node must wrap a level-2 circuit and output a level-1 circuit.

The *primitive node* construct (node) lifts a user-defined scalar function to a stream operator, providing a generic extension point for core computations. Commonly used generic operations (like identity, tuple projections, and group arithmetic) are separated into a class of *standard nodes*.

Table 1. Circuit syntax and denotational semantics. Constructs are polymorphic over stream level ℓ unless indicated.

Construct c	Type	Level	$\llbracket c \rrbracket$	$\text{plift}(c)$	$\text{incOpt}(c)$
<i>Primitive nodes</i>					
$\text{node}(f)$	$A \rightarrow B$	ℓ	$\uparrow^\ell f$	$\text{node}(f)$	$\text{incOpt}(\text{node}(f))$
<i>Standard nodes</i>					
$\text{const}(x)$	$A \rightarrow B$	ℓ	$\uparrow^\ell(\lambda_. x)$	$\text{const}(x)$	$\text{const}(x) \triangleright D$
id	$A \rightarrow A$	ℓ	id	id	id
fst	$A \times B \rightarrow A$	ℓ	$\uparrow^\ell \pi_1$	fst	fst
snd	$A \times B \rightarrow B$	ℓ	$\uparrow^\ell \pi_2$	snd	snd
add	$A \times A \rightarrow A$	ℓ	$\uparrow^\ell(+)$	add	add
sub	$A \times A \rightarrow A$	ℓ	$\uparrow^\ell(-)$	sub	sub
<i>Temporal nodes</i>					
delay	$A \rightarrow A$	ℓ	z^{-1}	$\uparrow \text{delay}$	delay
$\uparrow \text{delay}$	$A \rightarrow A$	2	$\uparrow z^{-1}$	–	$\uparrow \text{delay}$
<i>Structural combinators</i>					
$c_1 \triangleright c_2$	$A \rightarrow C$	ℓ	$\llbracket c_2 \rrbracket \circ \llbracket c_1 \rrbracket$	$\text{plift}(c_1) \triangleright \text{plift}(c_2)$	$\text{incOpt}(c_1) \triangleright \text{incOpt}(c_2)$
$c_1 \parallel c_2$	$A \rightarrow B \times C$	ℓ	$\lambda x. (\llbracket c_1 \rrbracket(x), \llbracket c_2 \rrbracket(x))$	$\text{plift}(c_1) \parallel \text{plift}(c_2)$	$\text{incOpt}(c_1) \parallel \text{incOpt}(c_2)$
<i>Feedback loops</i>					
$\text{loop}(c)$	$A \rightarrow B$	ℓ	$\lambda x. \text{fix } \alpha. \llbracket c \rrbracket(x, z^{-1}(\alpha))$	$\uparrow \text{loop}(\text{plift}(c))$	$\text{loop}(\text{incOpt}(c))$
$\uparrow \text{loop}(c)$	$A \rightarrow B$	2	$\lambda x. \text{fix } \alpha. \llbracket c \rrbracket(x, \uparrow z^{-1}(\alpha))$	–	$\uparrow \text{loop}(\text{incOpt}(c))$
<i>Level adapters</i>					
$\text{lifting}(c)$	$A \rightarrow B$	2 (c at 1)	$\uparrow \llbracket c \rrbracket$	–	$\Delta(\text{lifting}(c))$
$\text{bracket}(c)$	$A \rightarrow B$	1 (c at 2)	$\uparrow f \circ \llbracket c \rrbracket \circ \uparrow \delta_0$	$\text{lifting}(\text{bracket}(c))$	$\text{bracket}(\text{incOpt}(c))$

Several important circuits can be defined compositionally:

Definition 3.9 (Derived Circuit Constructs). The integration circuit $I = \text{loop}(\text{add})$ computes $\mathcal{I}$, and the differentiation circuit $D = (\text{id} \parallel \text{delay}) \triangleright \text{sub}$ computes $\mathcal{D}$. The unoptimized incremental form of a circuit c is given by $\Delta(c) = I \triangleright c \triangleright D$.

The denotational semantics $\llbracket c \rrbracket$ of a circuit $c : A \rightarrow B$ at stream level ℓ is a stream operator. For level 1, it is an operator $\mathcal{S}_A \rightarrow \mathcal{S}_B$. For level 2, it is $\mathcal{S}_{S_A} \rightarrow \mathcal{S}_{S_B}$. Here $\uparrow^1 f = \uparrow f$ denotes single lifting, and $\uparrow^2 f = \uparrow \uparrow f$ denotes double lifting.

Due to the bracket construct, which relies on the stream elimination operator f , the denotational semantics $\llbracket \cdot \rrbracket$ is a *partial* function. Whether it converges on a given input is the convergence detection problem we mentioned in §2.2.

We can now formalize the incremental optimization algorithm of DBSP as the following two syntactic functions:

Definition 3.10. plift lifts a level-1 circuit and pushes the lifting operator onto individual nodes. Its definition is in Table 1.

⁵ This restriction to the stream level is mainly for formalization convenience without loss of expressiveness. See §8 for more discussion on this point.

Definition 3.11. incOpt transforms a circuit into its optimized incremental version following the DBSP's syntax-guided algorithm. Its definition is in Table 1.

Note that for primitive nodes, $\text{incOpt}(\text{node}(f))$ is a customizable incremental form, which makes this algorithm quite extensible. The theory has provided an incremental form for a wide class of functions. For example, for a linear function f , the incremental form can be $\text{node}(f)$ itself. Even in the worst case, the incremental form can fall back to the unoptimized form $\Delta(\text{node}(f))$.

Another thing to notice is that $\text{incOpt}(\text{lifting}(c))$ is the unoptimized form, as the incremental operator Δ cannot go across lifting into the inner circuit. That's why we need the function plift to push the lifting operator into the inner circuit.

3.3 Example: Implementing and Incrementalizing the Datalog Query in DBSP

In this section, we present how DBSP expresses and incrementalizes the Datalog query example from §1 and §2.1 in three steps (corresponding to the three strategies in §2.1).

DBSP uses $\mathbb{Z}$ -sets [19] (or simply Zsets) as the underlying Abelian group to represent sets and relations. It can be simply understood as sets where each element has an integer count, which can be negative to represent deletion. In this paper, we will simply treat Zsets as their underlying sets for most cases.

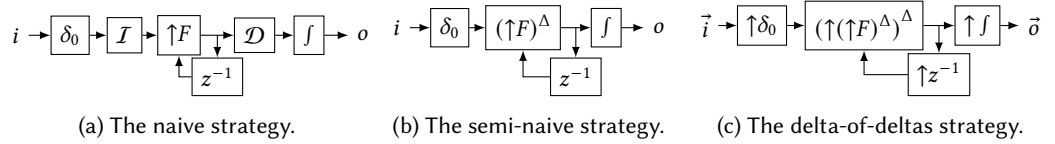


Fig. 6. The three circuits that arise when DBSP implements and incrementalizes a Datalog recursive query.

Recall that the recursive query is defined via a non-recursive query $F_E(R)$, or $F(E, R)$ (which we will simply write as F). DBSP will first compile it into some circuit and lift it to get a level-1 circuit c_0 whose denotation is $\uparrow F$. The recursive query Q asks for the fixpoint of iterative application of F on an input i . Q can be expressed as a circuit shown in Fig. 6a. The query body (the bracketed sub-circuit) can be expressed in our formal language as⁶

$$\text{datalogBody}(c_0) = \text{loop}(\Delta(c_0)) \quad (1)$$

We use the stream introduction operator δ_0 (Def. 3.7) and the integration operator $\mathcal{I}$ (Def. 3.6) to convert the input i into a constant stream of i and then feed it into $\uparrow F$. The feedback loop implements the iterative fixpoint computation we need. Its output stream α looks like following:

$$\alpha[0] = F(i, 0), \alpha[1] = F(i, \alpha[0]), \alpha[2] = F(i, \alpha[1]), \dots$$

If α is fixed after some point, then the differentiation operator $\mathcal{D}$ (Def. 3.6) converts it into a stream that is zero almost everywhere, which can then be aggregated by the stream elimination operator f (Def. 3.8) to produce the final output o . This corresponds to the naive strategy in §2.1.

Note that we can optimize $\uparrow F$ between $\mathcal{I}$ and $\mathcal{D}$ into $(\uparrow F)^\Delta$ by applying the DBSP's incremental optimization algorithm, naturally leading to the circuit implementing the semi-naive strategy shown in Fig. 6b. Its query body can be expressed as

$$\text{semiNaiveBody}(c_0) = \text{loop}(\text{incOpt}(c_0)) \quad (2)$$

⁶Technically, we cannot express the whole level-0 circuit itself in our formal language because of our syntactic restriction (see §8), but it suffices to express the query body as it encodes sufficient information.

We can further lift the whole circuit, push the lifting operator and apply the incremental optimization to transform it into the circuit shown in Fig. 6c, which implements the delta-of-deltas strategy. Its query body can be expressed as

$$\text{deltaOfDeltasBody}(c_0) = \uparrow \text{loop} \left(\text{incOpt} \left(\text{plift}(\text{incOpt}(c_0)) \right) \right) \quad (3)$$

and the whole circuit can be expressed as

$$\text{deltaOfDeltasQuery}(c_0) = \text{bracket}(\text{deltaOfDeltasBody}(c_0)) \quad (4)$$

4 The Fixpoint-Detection Problem and the Convergence-Detection Problem

As introduced in §3.1, the stream-elimination operator f is inherently a partial function: it is well-defined only if its input stream is *zero almost everywhere*. With standard Zset-based relational operators, DBSP is Turing-complete [6], meaning it is acceptable that some circuits containing f never converge to zero. However, the core challenge is at run time: even if f 's input does eventually converge to zero, how can the system safely detect this convergence by observing only a finite prefix of the stream? Determining this zero-almost-everywhere property at run time is the essence of the *Fixpoint Detection (FPD)* problem. In this section, we formalize the FPD problem and the *soundness* and *completeness* of fixpoint detectors.

Furthermore, in our language (§3.2), f operators are packaged inside bracketed circuits, and a complete circuit can contain multiple bracketed sub-circuits, much like a standard program can contain multiple while loops. Therefore, a circuit mathematically *converges* if and only if every bracketed sub-circuit reaches a fixpoint. We refer to this global detection problem as the *Convergence Detection* problem.

To formalize the FPD problem within our formal language, we introduce the following definition to characterize when a stream has converged to zero at a specific index:

Definition 4.1 (ZeroAfter). A stream $s \in S_A$ is *zero after an outer index* $n \in \mathbb{N}$ if:

$$\text{ZeroAfter}_1(s, n) := \forall m \geq n, s[m] = 0_A$$

A nested stream $s \in S_{S_A}$ is *zero after a 2D-index* $(m, n) \in \mathbb{N} \times \mathbb{N}$ if:

$$\text{ZeroAfter}_2(s, m, n) := \text{ZeroAfter}_1(s[m], n)$$

We also say that a nested stream $s \in S_{S_A}$ is *zero after an upper-bound stream* $\mathbf{b} \in S_{\mathbb{N}}$ if:

$$\text{ZeroAfter}_2(s, \mathbf{b}) := \forall i \in \mathbb{N}, \text{ZeroAfter}_2(s, i, \mathbf{b}[i])$$

An important fact is that ZeroAfter is a *level-indexed predicate* with the following properties:

- ZeroAfter₁ applies to any stream (i.e., stream of level $\ell \geq 1$), and accepts a 1D-index.
- ZeroAfter₂ is defined in terms of ZeroAfter₁. It applies only to nested streams (i.e., streams of level $\ell \geq 2$), and accepts a 2D-index or a stream of 1D-indices.

This inductive⁷ structure is a natural result of the stream-level hierarchy of a well-defined circuit. We will soon see more examples of such level-indexed definitions.

We also need to introduce another predicate to capture the first true value of a stream of Booleans.

Definition 4.2 (FirstTrue). A stream of Booleans $s \in S_{\mathbb{B}}$ encounters its first true value at index $n \in \mathbb{N}$, denoted $\text{FirstTrue}_1(s) = n$, if $s[n] = \text{true}$ and $\forall m < n, s[m] = \text{false}$. Similarly, we define $\text{FirstTrue}_2(s) = \mathbf{b}$ for $s \in S_{S_{\mathbb{B}}}$ and $\mathbf{b} \in S_{\mathbb{N}}$ as $\forall i \in \mathbb{N}, \text{FirstTrue}_1(s[i]) = \mathbf{b}[i]$.

⁷One might consider Def. 4.1 to not be inductive because it only refers to levels 1 and 2. However, this choice was deliberate because of the syntactic restriction we placed on stream level (see §3.2 and §8). If the stream level is generalized to any natural number, Def. 4.1 would be a standard inductive definition.



Fig. 7. The FPD problem for level-1 and level-2 circuits.

We can now formally state the FPD problem. Intuitively, the FPD problem is a matter of providing a fixpoint detector that specifies a runtime fixpoint-detection mechanism for a level-1 bracketed circuit c (as shown in Fig. 7a) or a level-2 bracketed circuit c (as shown in Fig. 7b). A fixpoint detector accepts a circuit c and an input to this circuit (which is either $\delta_0(x)$ or $\uparrow\delta_0(x)$), and returns a Boolean value for each entry of the input. When a fixpoint detector returns true for the first time in an iteration, the output of c should have converged to 0 in the corresponding iteration (recall that c should produce changes to the output that are integrated by f until convergence). We call this property *soundness* (see Eqns. (5) and (7) below). When the output has converged to 0, the fixpoint detector should eventually return true. We call this property *completeness* (see Eqns. (6) and (8) below).

Definition 4.3 (The FPD problem). Given a level-1 circuit c , a function $f_c : \mathcal{S}_A \rightarrow \mathcal{S}_{\mathbb{B}}$ is a *fixpoint detector* for c if for any $x \in A$ and $n \in \mathbb{N}$:

$$\text{FirstTrue}_1(f_c(\delta_0(x))) = n \implies \text{ZeroAfter}_1(\llbracket c \rrbracket(\delta_0(x)), n) \text{ and} \quad (5)$$

$$\text{ZeroAfter}_1(\llbracket c \rrbracket(\delta_0(x)), n) \implies \exists m, \text{FirstTrue}_1(f_c(\delta_0(x))) = m \quad (6)$$

Similarly, given a level-2 circuit c , a function $f_c : \mathcal{S}_{S_A} \rightarrow \mathcal{S}_{S_{\mathbb{B}}}$ is a *fixpoint detector* for c if for any $x \in \mathcal{S}_A$ and $\mathbf{b} \in \mathcal{S}_{\mathbb{N}}$:

$$\text{FirstTrue}_2(f_c(\uparrow\delta_0(x))) = \mathbf{b} \implies \text{ZeroAfter}_2(\llbracket c \rrbracket(\uparrow\delta_0(x)), \mathbf{b}) \text{ and} \quad (7)$$

$$\text{ZeroAfter}_2(\llbracket c \rrbracket(\uparrow\delta_0(x)), \mathbf{b}) \implies \exists \mathbf{b}', \text{FirstTrue}_2(f_c(\uparrow\delta_0(x))) = \mathbf{b}' \quad (8)$$

By changing FirstZero to another fixpoint specification or criterion (like our IntFP to be introduced in §5), Def. 4.3 can be adapted to formalize other FPD problems. Note that the FPD for a level-1 circuit c on some input x can be simply reduced to the FPD for the level-2 circuit $\uparrow c$ on stream input $\lambda_. x$. Consequently, level-2 FPD is a harder problem than level-1 FPD.

For the theory's global convergence specification, we define it as follows:

Definition 4.4 (External convergence (ExtConv)). For any circuit c and input x , we say c *externally converges*, denoted $\text{ExtConv}(c, x)$, if $\llbracket c \rrbracket(x)$ mathematically converges. Or equivalently, for any sub-circuit in the form of $\text{bracket}(c_0)$ and its corresponding input x_0 , $\exists \mathbf{b}, \text{ZeroAfter}_2(\llbracket c_0 \rrbracket(\uparrow\delta_0(x_0)), \mathbf{b})$.

It is easy to see why the external-convergence problem is just the global version of the FPD problem. Once we have a fixpoint detector for any circuit c , we immediately get a convergence detector for ExtConv by applying the fixpoint detector to every bracketed sub-circuit. Therefore, we will only focus on the FPD problem in what follows.

4.1 Example: The FirstZero Strategy and Its Limitation

We have already introduced the FirstZero strategy in Section 2.1, which is merely to stop when the first 0 value is encountered. We can formalize FirstZero by instantiating the f_c fixpoint-detector function from Def. 4.3. For a level-1 circuit c , it maps an input stream x to a Boolean stream: $f_c(x) = \lambda n. (\llbracket c \rrbracket(x)[n] = 0)$. For a level-2 circuit c , it maps an input nested stream x to a nested Boolean stream: $f_c(x) = \lambda i. \lambda n. (\llbracket c \rrbracket(x)[i][n] = 0)$.

Following the intuitive idea in §2.1, we have established the following theorems for the soundness and completeness of FirstZero for the naive and semi-naive strategy, and the unsoundness of FirstZero for the delta-of-deltas strategy.

Theorem 4.5. The FirstZero strategy is a sound and complete fixpoint detector for the naive query body $\text{datalogBody}(c_0)$ (Eqn. (1)) and the semi-naive query body $\text{semiNaiveBody}(c_0)$ (Eqn. (2)), where c_0 can represent any circuit implementing a lifted scalar query.

Theorem 4.6. The FirstZero strategy is not a sound fixpoint detector for the delta-of-deltas query body $\text{deltaOfDeltasBody}(c_0)$ (Equation 3), where c_0 is the circuit implementing the lifted non-recursive query $F_E(R)$ from §1.

4.2 Impossibility of Exact Fixpoint Detection

Can another strategy overcome FirstZero's failure and solve FPD exactly for every circuit? The following theorem shows the answer is no. We assume the choice of primitive nodes makes DBSP *Turing-complete*: DBSP circuits can express arbitrary partial-recursive scalar functions. This assumption is mild in practice: standard Zset primitive nodes together with DBSP's recursive constructs suffice [6, §7.2]. Without loss of generality, we also assume the base type is $\mathbb{Z}$.

Theorem 4.7. If the choice of primitive nodes makes DBSP Turing-complete, there exists a fixed level-1 circuit c with no computable sound and complete fixpoint detector.

PROOF. We show that a computable sound and complete detector would recognize⁸ the complement of the halting problem, which is not recursively enumerable. We construct a fixed circuit whose output is eventually zero exactly for inputs that encode nonhalting programs. Let $h(e, k)$ be 1 if the program that e encodes halts on input 0 within k steps of a fixed universal evaluator, and 0 otherwise. Since h is total and primitive recursive [7, §5.2], Turing-completeness ensures there is a level-1 circuit E with $\llbracket E \rrbracket = \uparrow h$. Define $c := (I \parallel (\text{const}(1) \triangleright I)) \triangleright E$. On $\delta_0(e)$, the two branches supply e and $n + 1$, giving $\llbracket c \rrbracket(\delta_0(e))[n] = h(e, n + 1)$. Assume there is a computable sound and complete detector for c and run the detector on $\delta_0(e)$ until its first true output. By completeness, this procedure terminates if e does not halt; by soundness, it runs forever if e halts. This would recognize the complement of the halting problem, a contradiction. $\square$

The theorem only rules out exact fixpoint detection for *arbitrary circuits*, but it does not rule out sound and complete detectors for *certain circuit subclasses of interest*.⁹ It motivates the sound sufficient criterion IntConv developed in the next section. In §7, we prove that IntConv is also complete for regular circuits, including Datalog and while queries, and then extend this guarantee to their incrementally optimized forms.

5 A Sound Convergence Criterion: IntConv

In this section, we introduce *internal convergence* (IntConv) as a sound sufficient condition for ExtConv. (In §6, we give an implementable detector for the IntConv criterion.) Following the “local-global” structure of the problem, we will first define *internal fixpoint* (IntFP) as a sound sufficient condition for an external fixpoint, and then apply it to all bracketed sub-circuits to formulate the IntConv criterion. At the end of this section, we give a qualitative data-dependency example in which relying on ExtConv without IntConv requires recursive trace-backs or additional retained state. The example provides a theoretical argument for an implementation benefit of IntConv.

Before introducing IntFP, we first introduce a natural generalization of ZeroAfter called FixAfter, which indicates that a stream becomes fixed to a constant value rather than to 0_A .

Definition 5.1 (FixAfter). Like ZeroAfter, FixAfter is a level-indexed predicate defined identically in its level-1, level-2, and vectorized forms, except that its base case $\text{FixAfter}_1(s, n)$ requires $\forall m \geq$

⁸In computability theory, an algorithm recognizes a set if the algorithm eventually accepts every input in the set and never accepts inputs outside it; on those other inputs, it may run forever.

⁹For example, this construction does not establish an impossibility result for incrementalizations of total computable queries, because the query $f(\llbracket c \rrbracket(\delta_0(e)))$ is defined exactly for nonhalting programs.

$n, s[m] = s[n]$. Note that if s is a nested stream $\mathcal{S}_{S_A}$, this means repeating the exact same *inner stream* $s[n]$ from the outer index n onwards.

Then, we reframe the FPD problem using a new level-indexed predicate called external fixpoint (ExtFP). Intuitively, ExtFP characterizes that a circuit reaches an external fixpoint at a given index, without looking into the internal circuit structure. Formally,

Definition 5.2 (External fixpoint (ExtFP)). Given any circuit c (either level-1 or level-2), an input stream x , and an index $n \in \mathbb{N}$, we say that c reaches an *external fixpoint* across the first time dimension at n , denoted by $\text{ExtFP}_1(c, x, n)$, if both the input and the denoted output of c are fixed after the outer index n :

$$\text{ExtFP}_1(c, x, n) := \text{FixAfter}_1(x, n) \wedge \text{FixAfter}_1(\llbracket c \rrbracket(x), n)$$

For a level-2 circuit c , a nested input stream $x \in \mathcal{S}_{S_A}$, an outer index $m \in \mathbb{N}$, and an inner index $n \in \mathbb{N}$, the external fixpoint across the second time dimension is defined as:

$$\text{ExtFP}_2(c, x, m, n) := \text{FixAfter}_2(x, m, n) \wedge \text{FixAfter}_2(\llbracket c \rrbracket(x), m, n)$$

$\text{ExtFP}_2(c, x, \mathbf{b})$ for $\mathbf{b} \in \mathcal{S}_{\mathbb{N}}$ is naturally defined as $\forall i \in \mathbb{N}, \text{ExtFP}_2(c, x, i, \mathbf{b}[i])$.

With this definition, the FPD problem for a level-1 circuit c on input x essentially asks to detect an index n where $\text{ExtFP}_1(c, \delta_0(x), n)$ holds and the fixed output is 0. Similarly, the FPD problem for a level-2 circuit c on nested input x asks to detect a bound stream $\mathbf{b}$ where $\text{ExtFP}_2(c, \uparrow\delta_0(x), \mathbf{b})$ holds and the fixed outputs are 0.

As we proved in §4.2, exact FPD is impossible for sufficiently expressive circuits, and hence so is detecting ExtFP in full generality. The Feldera implementation of DBSP [14] uses internal-state stability to detect fixpoints. Each runtime operator provides a scope-indexed fixed-point check intended to report whether its output will remain stable if its input does, and the circuit combines the checks of its operators. Thus, the operational idea of detecting internal-state stability is established implementation practice rather than a novel contribution of this paper. The cited implementation specifies and tests this runtime contract, but does not provide a formal proof that the circuit-wide check implies denotational convergence. To bridge the gap between this runtime strategy and the denotational DBSP theory, we give the strategy a rigorous, declarative account through a criterion called *internal fixpoint* (IntFP). This formal criterion allows us to prove precisely this soundness connection, and later establish completeness for a large class of circuits. We discuss the relationship with the Feldera implementation further in §9.

Definition 5.3 (Internal fixpoint (IntFP)). Intuitively, IntFP means that all internal nodes (i.e., individual components) of a circuit have reached an ExtFP at a specific index. Formally, for any circuit c , an input stream x , and an index $n \in \mathbb{N}$, $\text{IntFP}_1(c, x, n)$ is defined by structural induction on c over the categories in Table 1:

- **Primitive, Standard and Temporal Nodes:** Inherit the ExtFP_1 condition. That is, for any such node c , $\text{IntFP}_1(c, x, n) := \text{ExtFP}_1(c, x, n)$.
- **Structural Combinators:** Require IntFP_1 for all sub-circuits. That is

$$\text{IntFP}_1(c_1 \triangleright c_2, x, n) := \text{IntFP}_1(c_1, x, n) \wedge \text{IntFP}_1(c_2, \llbracket c_1 \rrbracket(x), n)$$

and

$$\text{IntFP}_1(c_1 \parallel c_2, x, n) := \text{IntFP}_1(c_1, x, n) \wedge \text{IntFP}_1(c_2, x, n)$$

- **Feedback Loops:** Require IntFP_1 for the loop body on the input tuple containing the delayed output. That is

$$\text{IntFP}_1(\text{loop}(c), x, n) := \text{IntFP}_1(c, (x, z^{-1}(\llbracket \text{loop}(c) \rrbracket(x))), n)$$

and similarly

$$\text{IntFP}_1(\uparrow\text{loop}(c), x, n) := \text{IntFP}_1(c, (x, \uparrow z^{-1}(\llbracket \uparrow\text{loop}(c) \rrbracket(x))), n)$$

- **Level Adapters:** For $\text{bracket}(c)$, IntFP_1 delegates to the level-2 internal circuit c :

$$\text{IntFP}_1(\text{bracket}(c), x, n) := \text{IntFP}_1(c, \uparrow\delta_0(x), n)$$

For the level-1 circuit $\text{lifting}(c)$, because it models an inner circuit that lacks states across outer iterations, the internal fixpoint naturally degenerates into the external fixpoint:

$$\text{IntFP}_1(\text{lifting}(c), x, n) := \text{ExtFP}_1(\text{lifting}(c), x, n)$$

The predicate $\text{IntFP}_2(c, x, m, n)$ for the second time dimension is defined for level-2 circuits only. It is mostly defined by uniformly replacing ExtFP_1 with ExtFP_2 and IntFP_1 with IntFP_2 , with a notable exception. For the nested lifting circuit $\text{lifting}(c)$, IntFP relies on the first dimension predicate IntFP_1 of the underlying level-1 circuit:

$$\text{IntFP}_2(\text{lifting}(c), x, m, n) := \text{IntFP}_1(c, x[m], n)$$

The vectorized version is naturally defined as $\text{IntFP}_2(c, x, \mathbf{b}) := \forall i \in \mathbb{N}, \text{IntFP}_2(c, x, i, \mathbf{b}[i])$.

The definition of IntFP is both structural and level-indexed. For a bracketed circuit, it applies IntFP to the enclosed circuit, adapting the input to its stream level. Because the same rules apply recursively to sub-circuits, the definition also handles nested brackets.

One of the most important properties of IntFP is that it is a sound sufficient condition for ExtFP . That is, if a circuit reaches an internal fixpoint, it must have also reached an external fixpoint.

Theorem 5.4 (Soundness of IntFP). For any well-formed circuit c , input x , and index n , we have:

$$\text{IntFP}_1(c, x, n) \implies \text{ExtFP}_1(c, x, n)$$

Similarly, for any level-2 circuit c , nested input x , and indices m, n :

$$\text{IntFP}_2(c, x, m, n) \implies \text{ExtFP}_2(c, x, m, n)$$

PROOF. We first prove the property for the level index 1, and then prove the property for the level 2 based on that for level 1. This can also be seen as induction on the level index of the predicate. $\square$

Finally, by applying the local IntFP criterion to all bracketed sub-circuits, and adding that the fixed output is 0, we define our global convergence criterion called *internal convergence* (IntConv).

Definition 5.5 (Internal convergence (IntConv)). We say that a circuit c internally converges on input x , denoted $\text{IntConv}(c, x)$, if for any sub-circuit in the form of $\text{bracket}(c_0)$ and its input x_0 ,

$$\exists \mathbf{b}, \text{IntFP}_2(c_0, \uparrow\delta_0(x_0), \mathbf{b}) \wedge \forall i, \llbracket c_0 \rrbracket(\uparrow\delta_0(x_0))[i][\mathbf{b}[i]] = 0$$

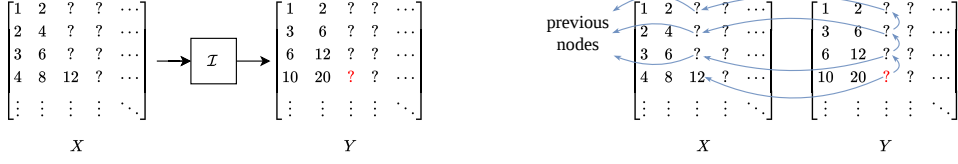
By Theorem 5.4, the definition of IntConv will be equivalent with $\forall i, \llbracket c_0 \rrbracket(\uparrow\delta_0(x_0))[i][\mathbf{b}[i]] = 0$ replaced by $\text{ZeroAfter}_2(\llbracket c_0 \rrbracket(\uparrow\delta_0(x_0)), \mathbf{b})$. This immediately implies that IntConv is a sound sufficient condition for ExtConv .

Theorem 5.6 (Soundness of IntConv). For any well-formed circuit c and input x , we have

$$\text{IntConv}(c, x) \implies \text{ExtConv}(c, x)$$

These soundness implications are one-way in general: ExtFP need not imply IntFP , and ExtConv need not imply IntConv . In §7, we identify circuit classes for which the corresponding converse does hold.

Why use IntConv rather than a weaker sound criterion for the theoretical specification ExtConv ? The following example isolates one implementation benefit. For this circuit, an evaluator that relies only on ExtConv lacks a guarantee that earlier internal values have stabilized. It must therefore



(a) An $\mathcal{I}$ operator inside a bracketed sub-circuit. The red ? is the value we want to evaluate.

(b) Data dependency in this example. An arrow from a to b denotes that computing a requires b .

Fig. 8. A data-dependency cost that can arise when evaluation relies on ExtConv without IntConv.

either recompute a recursive dependency chain or retain the historical values needed to answer later demands. IntConv supplies the missing stability guarantee.

Consider such a circuit satisfying ExtConv but not IntConv, with an integration operator $\mathcal{I}$ inside a bracketed sub-circuit, as shown in Fig. 8. Its input X and output Y are both nested streams. Assume ExtConv is detected soundly. Suppose that evaluation is currently at $(4, 3)$ (we use 1-based indexing)—namely, the third inner iteration of the fourth outer iteration—and that the first three outer iterations reached external convergence after two inner iterations, while the fourth did not. Now $X[4][3] = 12$ is ready.

Fig. 8b shows the data dependency in this example. To evaluate $Y[4][3]$ correctly, we need $Y[4][3] = Y[3][3] + X[4][3]$. But because the circuit satisfies ExtConv without satisfying IntConv, external convergence does not guarantee internal convergence. Consequently, we cannot directly conclude what $Y[3][3]$ is. Computing $Y[3][3]$ may require $Y[2][3]$ and $X[3][3]$, both unknown; computing $X[3][3]$ can recursively force upstream re-evaluation, potentially all the way back to $\uparrow\delta_0$, which initiated the iterative computation.

Thus, under a strategy that does not retain all of this history, a later outer iteration that needs more inner iterations than earlier ones can trigger recursive recomputation. Retaining the history instead trades that recomputation for additional state, potentially across an unbounded number of outer iterations.

For this example, IntConv avoids that tradeoff. Because IntConv guarantees that all internal nodes reach an internal fixpoint, values like $Y[3][3]$ are mathematically guaranteed to be stabilized values (e.g., $Y[3][3] = Y[3][2]$ in this example), which should already be stored in the state. No recursive trace-back is needed, and state management stays simple: in this example, $\mathcal{I}$ only needs to store its output list (all output until convergence) from the last outer iteration. This analysis is qualitative: it neither establishes a general lower bound for every criterion weaker than IntConv nor measures the overhead of a concrete implementation.

6 The StFP Detector and the Global IntConv Detector

According to Def. 5.5, the global IntConv detector checks every bracketed sub-circuit with a local IntFP detector and also checks that its fixed output is 0. We construct the local detector in two steps: the StFP detector checks local state stability, and a fixed-input check turns its result into IntFP detection. Thus, the global IntConv detector combines the StFP detector with fixed-input and zero-output checks across all bracketed sub-circuits.

To detect the IntFP, the Feldera implementation for the DBSP runtime uses a scope-indexed internal-state-stability strategy that we call “FirstStateStable”, as opposed to FirstZero. The idea in our core calculus is as follows: most nodes are stateless, and only z^{-1} and $\uparrow z^{-1}$ are stateful. For stateless nodes, their output is solely determined by their input. For stateful z^{-1} and $\uparrow z^{-1}$, their

output in an execution step is solely determined by their state and unrelated to the input of the current step, which means we can predict all stateful nodes' outputs at the end of the previous step. Therefore, at the end of a step, if we can tell that the outputs of all stateful nodes of a circuit are the same as the previous step, and that the input to the whole circuit is fixed after this step (which is easily decidable for input like $\delta_0(x)$), then we can conclude that the circuit has reached an IntFP.

To formalize this idea, a challenge is that for simplicity, our theory only specifies the denotational semantics of DBSP circuits as pure mathematical functions, and has no formal concepts of "states". After all, although we have some model in mind of how states are implemented, ultimately it is a representation choice in the implementation. Luckily, there is an important assumption that holds for any reasonable implementation of states: *a circuit's state at a given point is determined by its input up to that point*. Therefore, we can always formalize properties of a circuit's state in terms of its input up to that point.

Following this idea, we first make precise how to continue an observed input by holding it fixed. For a stream x and an index n , let

$$\text{letFixed}_1(x, n)[i] := \begin{cases} x[i] & i < n, \\ x[n] & i \geq n, \end{cases}$$

and, for a nested stream x and indices m, n , let

$$\text{letFixed}_2(x, m, n)[i][j] := \begin{cases} x[m][n] & i = m \wedge j \geq n, \\ x[i][j] & \text{otherwise.} \end{cases}$$

Thus, letFixed_2 leaves every row other than m unchanged and replaces the suffix of row m beginning at n with the value $x[m][n]$.

We can now define the notion of *State Fixpoint (StFP)* as a semantic counterpart of the circuit-state property used by the FirstStateStable strategy.

Definition 6.1 (State Fixpoint (StFP)). For any circuit c , input x , and index n , define

$$\text{StFP}_1(c, x, n) := \text{IntFP}_1(c, \text{letFixed}_1(x, n), n).$$

For any level-2 circuit c , nested input x , and indices m, n , define

$$\text{StFP}_2(c, x, m, n) := \text{IntFP}_2(c, \text{letFixed}_2(x, m, n), m, n).$$

Finally, for a bound stream $\mathbf{b} \in \mathcal{S}_{\mathbb{N}}$, the vectorized predicate is defined pointwise:

$$\text{StFP}_2(c, x, \mathbf{b}) := \forall i \in \mathbb{N}, \text{StFP}_2(c, x, i, \mathbf{b}[i]).$$

The fixed continuations agree with the observed input through the designated index and impose a fixed future from that index onward. Therefore, StFP asks whether the circuit's current state would be at an internal fixpoint if its future input remained stable. The following theorem states its precise connection to IntFP.

Theorem 6.2 (IntFP and StFP). For any circuit c ,

$$\forall x, n, \text{IntFP}_1(c, x, n) \iff (\text{StFP}_1(c, x, n) \wedge \text{FixAfter}_1(x, n))$$

and for any level-2 circuit c ,

$$\forall x, m, n, \text{IntFP}_2(c, x, m, n) \iff (\text{StFP}_2(c, x, m, n) \wedge \text{FixAfter}_2(x, m, n))$$

The pointwise definitions immediately give the vectorized form

$$\forall x, \mathbf{b}, \text{IntFP}_2(c, x, \mathbf{b}) \iff (\text{StFP}_2(c, x, \mathbf{b}) \wedge \text{FixAfter}_2(x, \mathbf{b})).$$

Because it is easy to detect whether $\uparrow\delta_0(x)$ (the input to the target bracketed circuit) is fixed after any index (m, n) , it suffices to detect the StFP.

We now give a high-level runtime algorithm to detect the StFP for *any* circuit c and input x . While StFP provides the semantic formalization of FirstStateStable in our denotational model, this detector provides its algorithmic formalization. As before, we simulate computation of the state at some point by computation of input up to that point. We will also explain what a natural implementation would probably do for each case. See §9 for more discussion of the Feldera implementation.

Definition 6.3 (StFP Detector). We define Detector_1 , the level-1 StFP detector that accepts a circuit c , stream input x , and an index n , and returns a boolean, as follows:

- **Primitive and Standard Nodes:** Return true because they are stateless.

$$\text{Detector}_1(c, x, n) := \text{true}$$

- **Temporal Nodes:** For delay, it checks whether the new state (the current input) matches the stored state (the last input). Note that for level-2 delay, we technically need to compare two infinite streams. However, under IntConv, the stream of last input is always fixed after some point, so it can be stored in a finite state (see the example at the end of §5).

$$\text{Detector}_1(\text{delay}, x, n) := (x(n) = z^{-1}(x)(n))$$

For $\uparrow\text{delay}$, because its states are only updated within the inner time dimension, it has no states across the outer time dimension, so we always return true.

$$\text{Detector}_1(\uparrow\text{delay}, x, n) := \text{true}$$

- **Structural Combinators:** Call Detector_1 for its sub-circuits. That is

$$\text{Detector}_1(c_1 \triangleright c_2, x, n) := \text{Detector}_1(c_1, x, n) \wedge \text{Detector}_1(c_2, \llbracket c_1 \rrbracket(x), n)$$

and

$$\text{Detector}_1(c_1 \parallel c_2, x, n) := \text{Detector}_1(c_1, x, n) \wedge \text{Detector}_1(c_2, x, n)$$

- **Feedback Loops:** For $\text{loop}(c)$, it calls the detector on the loop body and checks whether the new state of the z^{-1} on the back edge (the current output) matches the stored state (the delayed output). o denotes the loop output $\llbracket \text{loop}(c) \rrbracket(x)$.

$$\text{Detector}_1(\text{loop}(c), x, n) := \text{Detector}_1(c, (x, z^{-1}(o)), n) \wedge (o(n) = z^{-1}(o)(n))$$

For $\uparrow\text{loop}(c)$, the $\uparrow z^{-1}$ on the back edge has no states across outer iterations, so it only calls the detector on the inner circuit. In the following, o denotes $\llbracket \uparrow\text{loop}(c) \rrbracket(x)$.

$$\text{Detector}_1(\uparrow\text{loop}(c), x, n) := \text{Detector}_1(c, (x, \uparrow z^{-1}(o)), n)$$

- **Level Adapters:** For $\text{bracket}(c)$, it delegates to the internal circuit c :

$$\text{Detector}_1(\text{bracket}(c), x, n) := \text{Detector}_1(c, \uparrow\delta_0(x), n)$$

For $\text{lifting}(c)$, it lacks states across outer iterations:

$$\text{Detector}_1(\text{lifting}(c), x, n) := \text{true}$$

Similarly, we define Detector_2 as the level-2 StFP detector that accepts a level-2 circuit c , a nested stream input x , an index (m, n) , and returns a boolean. For standard and primitive nodes and structural combinators, it is defined by uniformly replacing Detector_1 with Detector_2 and the index n with (m, n) . For other cases:

- **Temporal Nodes:** For delay, it detects whether the delayed input from the last outer iteration has reached its fixpoint, which should have been stored in the state at the end of the last outer iteration (again, refer to the example at the end of §5):

$$\text{Detector}_2(\text{delay}, x, m, n) := \text{FixAfter}_2(z^{-1}(x), m, n)$$

For $\uparrow\text{delay}$, the condition drops to just checking the state along the inner dimension:

$$\text{Detector}_2(\uparrow\text{delay}, x, m, n) := (x(m, n) = \uparrow z^{-1}(x)(m, n))$$

- **Feedback Loops:** For $\text{loop}(c)$, similar to delay, it calls the detector on the internal circuit and checks whether the delayed output from the last outer iteration has reached its fixpoint. In the following, o denotes $\llbracket \text{loop}(c) \rrbracket(x)$.

$$\text{Detector}_2(\text{loop}(c), x, m, n) := \text{Detector}_2(c, (x, z^{-1}(o)), m, n) \wedge \text{FixAfter}_2(z^{-1}(o), m, n)$$

For $\uparrow\text{loop}(c)$, similar to $\uparrow\text{delay}$, it calls the detector on the internal circuit, and checks whether the current output matches the delayed output. In the following, o denotes $\llbracket \uparrow\text{loop}(c) \rrbracket(x)$.

$$\text{Detector}_2(\uparrow\text{loop}(c), x, m, n) := \text{Detector}_2(c, (x, \uparrow z^{-1}(o)), m, n) \wedge (o(m, n) = \uparrow z^{-1}(o)(m, n))$$

- **Level Adapters:** For $\text{lifting}(c)$, it delegates to the level-1 detector:

$$\text{Detector}_2(\text{lifting}(c), x, m, n) := \text{Detector}_1(c, x[m], n)$$

The FixAfter_2 conditions in the level-2 delay and loop clauses do not require predicting the future of the current outer iteration. Each is applied to a delayed stream, so its row at outer index m comes from the already completed outer iteration $m - 1$ (or is the zero row when $m = 0$). Before iteration m begins, the runtime has stored that row through a convergence bound B . If $n \geq B$, the bound immediately establishes FixAfter_2 ; otherwise, the runtime decides it by comparing the finitely many stored entries from n through B , because all later entries equal the stored final value.

Like the StFP predicate, the StFP detector is structural and level-indexed. Its clauses for structural combinators recurse into sub-circuits, while its level-adapter clauses delegate across circuit levels. Therefore, it applies compositionally to nested bracketed circuits rather than treating each recursive component as an isolated case.

We have proved the correctness of StFP via the following theorem:

Theorem 6.4 (Correctness of StFP Detector). For any circuit c ,

$$\forall x, n, \text{Detector}_1(c, x, n) \iff \text{StFP}_1(c, x, n)$$

and for any level-2 circuit c ,

$$\forall x, m, n, \text{Detector}_2(c, x, m, n) \iff \text{StFP}_2(c, x, m, n)$$

These immediately imply that the StFP detector is both sound and complete.

The StFP detector should be read primarily as a correctness-oriented general fallback detector. It establishes that StFP is decidable for arbitrary circuits in our core calculus. A fixed-input check turns it into a local IntFP detector; applying that detector with zero-output checks across all bracketed sub-circuits yields the sound and complete global IntConv detector and establishes that IntConv is decidable as well. A practical implementation may optimize these checks or use a specialized detector when one is justified. In particular, for Conv-complete classes, IntConv and ExtConv coincide, so a sound and complete ExtConv detector also detects IntConv. FirstZero for the naive and semi-naive circuits is one such specialized detector. Measuring the overhead of these alternatives in a complete system is future work.

7 IntConv is Complete for Useful Circuits

By Thm. 5.6, IntConv is a sound sufficient condition for ExtConv. A natural question is: for what circuits is IntConv *complete*? That is, when does mathematical convergence according to ExtConv also imply IntConv? For such circuits, a system following the IntConv criterion faithfully implements the theoretical specification. Moreover, because IntConv and ExtConv coincide on these circuits, a sound and complete ExtConv detector also detects IntConv. This permits specialized detectors such as FirstZero for the naive and semi-naive circuits instead of the general StFP detector from §6.

To answer this question, we introduce a key property called *convergence-complete* (Conv-complete) circuits. They are just the circuits mentioned above for which IntConv is complete. In §7.1, we use *regular circuits* as a name for the fragment generated by the standard DBSP constructs for lifted scalar computation, composition, Datalog recursion, and while recursion. The name lets us state a uniform completeness result for this existing recursive sublanguage: all regular circuits are Conv-complete (Thm. 7.9). Moreover, in §7.2, we prove that the DBSP's incremental optimization (plift and incOpt) preserves Conv-completeness (Thm. 7.12 and Thm. 7.19). As a result, all regular circuits and their incrementally optimized versions are Conv-complete.

Along with Conv-completeness, its local version FP-completeness is important for us to establish these results. We define them as follows:

Definition 7.1 (Conv-complete Circuit). A circuit c is *Conv-complete* iff for every input x ,

$$\text{ExtConv}(c, x) \implies \text{IntConv}(c, x).$$

Definition 7.2 (FP-complete Circuit). Let c be a well-formed circuit. We say c is *FP-complete* if:

- **Level-1 completeness:** for every input x and index n , if $\text{ExtConv}(c, x)$ and $\text{ExtFP}_1(c, x, n)$ hold, then there exists $n' \in \mathbb{N}$ such that $\text{IntFP}_1(c, x, n')$ holds.
- **Level-2 completeness (for level-2 circuits):** for every nested stream input x and stream b , if $\text{ExtConv}(c, x)$ and $\text{ExtFP}_2(c, x, b)$ hold, then there exists a stream b' such that $\text{IntFP}_2(c, x, b')$ holds.

We have established some compositional properties of Conv-completeness and FP-completeness. In particular, Conv-completeness is closed under all circuit constructs except for bracket, while FP-completeness is closed under all circuit constructs except for bracket and sequential composition seq. These results are useful for us to establish the important results in this section.

7.1 Regular Circuits Are Conv-complete

Recall that we can use the DBSP circuit shown in Fig. 6a to express a Datalog-style recursive query. We can express it in our formal language as follows.

Definition 7.3 (Datalog Recursive Query). For a level-1 circuit c_0 that represents a lifted scalar query, the query body (the circuit between δ_0 and f) of a Datalog-style recursive query is (the same as Eqn. (1))

$$\text{datalogBody}(c_0) := \Delta(\text{loop}(c_0))$$

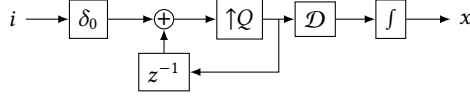
and the streaming query (the lifted version of the whole circuit) is

$$\text{datalogQuery}(c_0) := \text{bracket}(\text{lifting}(\text{datalogBody}(c_0)))$$

Beyond recursive relational queries, the following *while* program is used to show the Turing-completeness of DBSP [6], where Q is an arbitrary scalar function:

```
x := i
while (x != Q(x))
  x := Q(x)
```

which can be implemented in DBSP as:



In our formal language:

Definition 7.4 (While Loop Programs). For a level-1 circuit c_0 representing a lifted scalar function, the query body of a while loop program is

$$\text{whileBody}(c_0) := \text{loop}(\text{add} \triangleright c_0) \triangleright D$$

and the streaming query is

$$\text{whileQuery}(c_0) := \text{bracket}(\text{lifting}(\text{whileBody}(c_0)))$$

Both query constructors above belong to the standard DBSP recursive sublanguage. Their results are level-1 circuits representing lifted scalar functions, so they can be composed or placed inside further recursive computations to form nested loops. We give the name *regular circuits* to the sublanguage generated by these constructs so that we can state and prove FP- and Conv-completeness uniformly. We use *lifted scalar circuit* for the base circuits that implement lifted scalar or relational queries.

Definition 7.5 (Regular Circuit). *Regular Circuits*, denoted by `RegularCkt`, is the class of level-1 circuits defined inductively as follows:

- **Lifted Scalar Circuit:** Any level-1 circuit composed (both sequentially and in parallel) of only primitive and standard nodes is a *lifted scalar circuit*, which is also a regular circuit.
- **Sequence and Parallel Composition:** If c_1 and c_2 are regular circuits, their sequential composition $c_1 \triangleright c_2$ and parallel composition $c_1 \parallel c_2$ are regular circuits.
- **Datalog Query:** Given a regular circuit c_0 , $\text{datalogQuery}(c_0)$ is a regular circuit.
- **While Loop:** Given a regular circuit c_0 , $\text{whileQuery}(c_0)$ is a regular circuit.

Using the inductive structure, it is easy to show that all regular circuits are FP-complete:

Lemma 7.6 (Regular Circuits are FP-complete). Any regular circuit c is FP-complete. In particular, for any x, n , $\text{ExtFP}_1(c, x, n) \implies \text{IntFP}_1(c, x, n)$.

We proceed to show that Datalog queries are Conv-complete. We first show that datalogBody satisfies a variant of FP-completeness tailored for a recursive query body, whose input must be in the form of $\delta_0(x)$ and the fixed output is zero.

Lemma 7.7 (The iteration bound of datalogBody). If c_0 is a Conv-complete regular circuit, let $c := \text{datalogBody}(c_0)$, for any scalar value x and index n , we have

$$\text{ExtConv}(c, \delta_0(x)) \wedge \text{ExtFP}_1(c, \delta_0(x), n) \wedge \llbracket c \rrbracket(\delta_0(x))[n] = 0 \implies \text{IntFP}_1(c, \delta_0(x), n + 1)$$

This lemma gives the concrete upper bound $n + 1$ on the iteration at which IntFP holds. It also allows us to prove datalogQuery is Conv-complete.

Lemma 7.8. If c_0 is a Conv-complete regular circuit, then $\text{datalogQuery}(c_0)$ is Conv-complete.

We have proved similar lemmas to establish the same iteration bound $n + 1$ for whileBody and the Conv-completeness of whileQuery . These lemmas allow us to prove that regular circuits are Conv-complete by induction on the structure of regular circuits.

Theorem 7.9 (Regular Circuits are Conv-complete). Any regular circuit is Conv-complete.

From the proof above, we can see that the class of regular circuits is highly extensible. We can freely add more recursive circuit constructs similar to datalogBody and whileBody while preserving Conv-completeness, as long as we can establish the variant of FP-completeness in Thm. 7.7 for the new constructors.

7.2 Conv-complete Circuits Are Closed under Incremental Optimization

As we can see in our example, the standard incremental optimization of DBSP can be represented by two syntactic transformations applied to a non-incremental circuit: *plift* to push the lifting operator onto individual nodes and *incOpt* to apply compositional incremental rules. In this section, we aim to show that *plift* and *incOpt* preserve Conv-completeness. To do this, we establish their correctness theorems to show how they transform the denotation, the IntFP, and the IntConv of the original circuit. The IntFP results track convergence bounds through these transformations; they are semantic guarantees rather than measurements of runtime cost.

To formally state the correctness of *plift*, we introduce the notion of *strong semantic equivalence* that requires the equivalence of both external denotation and internal convergence.

Definition 7.10 (Strong Semantic Equivalence). For any circuit c_1 and c_2 , we define the strong semantic equivalence $c_1 \approx c_2$ as follows:

$$\forall x, \llbracket c_1 \rrbracket(x) = \llbracket c_2 \rrbracket(x) \wedge (\text{IntConv}(c_1, x) \iff \text{IntConv}(c_2, x))$$

Note that this implicitly implies that $\text{ExtConv}(c_1, x) \iff \text{ExtConv}(c_2, x)$ because $\llbracket c \rrbracket$ is partial.

Based on this definition, we have re-established important equivalence lemmas in the DBSP theory, such as the chain rule $\Delta(c_1 \triangleright c_2) \simeq \Delta(c_1) \triangleright \Delta(c_2)$ and the lifting cycle rule $\text{lifting}(\text{loop}(c)) \simeq \uparrow \text{loop}(\text{lifting}(c))$. Using these, we prove the correctness of *plift* as follows. It is a straightforward induction on the structure of c .

Theorem 7.11 (Correctness of *plift*). For any level-1 circuit c ,

- (1) (Denotation and IntConv) $\text{lifting}(c) \simeq \text{plift}(c)$.
- (2) (IntFP₁) For any x and n , $\text{ExtFP}_1(\text{plift}(c), x, n) \implies \text{IntFP}_1(\text{plift}(c), x, n)$.
- (3) (IntFP₂) For any x and $m, n \in \mathbb{N}$, $\text{IntFP}_1(c, x[m], n) \implies \text{IntFP}_2(\text{plift}(c), x, m, n)$.

This allows us to prove that *plift* preserves both Conv-completeness and FP-completeness.

Theorem 7.12 (*plift* preserves Conv-completeness and FP-completeness). For any level-1 circuit c , if c is Conv-complete, then $\text{plift}(c)$ is Conv-complete. If c is FP-complete, then $\text{plift}(c)$ is FP-complete.

For *incOpt*, the situation is more tricky. In particular, unlike $\text{lifting}(c)$ and $\text{plift}(c)$, $\Delta(c)$ and $\text{incOpt}(c)$ may have different IntConv behavior; just consider $\text{node}(\text{id})$ and $\Delta(\text{node}(\text{id}))$. This motivates us to introduce *weak semantic equivalence*, which only requires the equivalence of external denotation but not internal convergence.

Definition 7.13 (Weak Semantic Equivalence). For any circuit c_1 and c_2 , we define the weak semantic equivalence $c_1 \approx c_2$ as $\forall x, \llbracket c_1 \rrbracket(x) = \llbracket c_2 \rrbracket(x)$. Note that this implies that $\text{ExtConv}(c_1, x) \iff \text{ExtConv}(c_2, x)$ since $\llbracket c \rrbracket$ is a partial function.

Besides denotation, how *incOpt* transforms IntFP is also tricky. To see this, consider the simplest circuit *id*, whose optimized incremental form is itself. Although the two forms are the same circuit, their corresponding input is different, because the input x to the original form corresponds to the input $\mathcal{D}(x)$ to the incremental form. Because x and $\mathcal{D}(x)$ have different FixAfter indices in general, the two forms have different IntFP in general. This observation suggests that the formulation of the IntFP-preservation property that we desire for *incOpt* will be non-trivial.

An important observation is that although x and $\mathcal{D}(x)$ may have different FixAfter indices, they do have a connection as follows:

Proposition 7.14. For any stream s and $n \in \mathbb{N}$, $\text{FixAfter}_1(s, n) \iff \text{ZeroAfter}_1(\mathcal{D}(s), n + 1)$, and for any nested stream s and $\mathbf{b} \in \mathbb{S}_{\mathbb{N}}$, $\text{FixAfter}_2(s, \mathbf{b}) \implies \text{FixAfter}_2(\mathcal{D}(s), \uparrow \max(\mathbf{b}, z^{-1}(\mathbf{b})))$.

Note that *ZeroAfter* implies *FixAfter*. This inspires us to raise a natural question: *since input streams of c and $\text{incOpt}(c)$ have such FixAfter index relation, will all other intermediate streams share the same relation?* If so, as a result, IntConv should also be preserved. We formalize this idea as the following *preservation* relation.

Definition 7.15 (IntFP and IntConv Preservation Relation). For any circuit c_1 and c_2 , we say that c_1 's IntFP₁ is preserved by c_2 , denoted by $c_1 \rightsquigarrow_1 c_2$, if

$$\forall x, n, (\text{ExtConv}(c_1, x) \wedge \text{IntFP}_1(c_1, x, n)) \implies \text{IntFP}_1(c_2, \mathcal{D}(x), n + 1)$$

For level-2 circuit c_1 and c_2 , we say that c_1 's IntFP₂ is preserved by c_2 , denoted by $c_1 \rightsquigarrow_2 c_2$, if

$$\forall x, \mathbf{b}, (\text{ExtConv}(c_1, x) \wedge \text{IntFP}_2(c_1, x, \mathbf{b})) \implies \text{IntFP}_2(c_2, \mathcal{D}(x), \uparrow \max(\mathbf{b}, z^{-1}(\mathbf{b})))$$

For any circuit c_1 and c_2 , we say that c_1 's IntConv is preserved by c_2 , denoted by $c_1 \rightsquigarrow_C c_2$, if

$$\forall x, \text{IntConv}(c_1, x) \implies \text{IntConv}(c_2, \mathcal{D}(x))$$

These allow us to formally define what we expect from a “good” incremental optimization:

Definition 7.16 (Sound Incremental Form). For a circuit c_1 and c_2 , we say that c_2 is a *sound incremental form* of c_1 if they satisfy the following four properties:

- (1) (Denotation) $\Delta(c_1) \approx c_2$.
- (2) (IntFP₁) $c_1 \rightsquigarrow_1 c_2$.
- (3) (IntFP₂) $c_1 \rightsquigarrow_2 c_2$ if c_1 is a level-2 circuit.
- (4) (IntConv) $c_1 \rightsquigarrow_C c_2$.

Another important challenge is that incOpt is a flexible and extensible algorithm, as different primitive nodes can have different and custom incremental optimization. We deal with this via a uniform assumption that incOpt transforms any primitive node into a sound incremental form. This is a strong but still practical assumption, as we have established that the following common incremental forms are all sound:

Proposition 7.17 (Common Sound Incremental Forms).

- (1) (Unoptimized) For any circuit c , $\Delta(c)$ is a sound incremental form of c .
- (2) (Linear) For any linear function f and $c = \text{node}(f)$, c is a sound incremental form of c .
- (3) (Bilinear) For any bilinear function f and $c = \text{node}(f)$, the DBSP's bilinear incremental optimization [6] of c , denoted $\text{bilinearOpt}(c)$, is a sound incremental form of c .

The unoptimized incremental form provides a general sound fallback for any primitive node, ensuring the robustness of incOpt. From now on, we may assume that incOpt transforms all its primitive nodes into a sound incremental form, and we can finally establish the correctness of incOpt as follows:

Theorem 7.18 (Correctness of incOpt). For any circuit c , $\text{incOpt}(c)$ is a sound incremental form of c .

This allows us to show that incOpt preserves Conv-completeness.

Theorem 7.19 (incOpt preserves Conv-completeness). For any circuit c , if c is Conv-complete, then $\text{incOpt}(c)$ is Conv-complete.

As a result, we can prove that the circuit implementing the delta-of-deltas strategy as shown in Fig. 6c is Conv-complete.

Proposition 7.20. If c_0 is a regular circuit, then $\text{optDatalogQuery}(c_0)$ (Equation 4) is Conv-complete.

8 Lean Formalization

All results in this paper have been formalized in Lean 4. We adapted the existing Lean 3 formalization of the DBSP theory [8] to Lean 4 as a foundation, which accounts for 2,095 lines of code. On top of this, we formalized our new theory in 11,748 additional lines of code, covering the core IntConv theory, and its application such as a Hoare logic framework and several Zsets circuit case studies. These counts use cloc and exclude blank lines and comments.

The restriction of the stream level to $\ell \in \{1, 2\}$ in our formal language (§3.2) represents a deliberate trade-off between formalization effort (especially with mechanized proofs) and theoretical generality. This range is sufficient to express arbitrarily nested recursive queries and the delta-of-deltas incremental optimization, while keeping the formal verification tractable. In principle, there is no fundamental obstacle to generalizing ℓ to arbitrary natural numbers as our theory has already been developed in an inductive style (the level-indexed predicate).

Use of AI. AI is used to assist with Lean development and artifact preparation. The authors carefully reviewed all definitions and theorem statements and take full responsibility for the paper, formalization, and accompanying artifacts.

9 Related Work

Incremental computation. Incremental computing is a paradigm for computing based on the idea of saving time by leveraging previous computational results. It has a long history, going back to the 1964 and 1967 papers of Lombardi [24, 25]. There have been contributions from many fields, including theory (dynamic graph algorithms [4, 28] and dynamic data structures [5, 22]), databases (view maintenance [10, 20]), programming languages (incremental lambda-calculus [15], incremental parsing [17], interactive program-development environments [30], incremental dataflow analysis [31], differential propagation in iterative fixpoint solvers [13], and finite-differencing transformations in optimizing compilers [27]), document-preparation systems [9], constraint solvers [16], truth-maintenance systems [12], and no doubt many others. For a recent survey of incremental computation, see [23].

Some of the work described above is only for limited models of computation. DBSP is a Turing-complete language. Because of its generality, DBSP could be applied to many of the application areas addressed by previous work, although it would not necessarily provide the same runtime guarantees of specialized results. However, many of these applications would need the ability to detect fixpoints of nested computations. The work described in this paper contributes to realizing the potential of DBSP by addressing an important semantic lacuna in the DBSP formalism.

More specifically, prior work has studied incremental recursive and fixpoint computation from complementary perspectives. Alvarez-Picallo et al. [3] develop a general theory of derivatives of fixpoints for incremental evaluation of recursive computations and Datalog. Abo Khamis et al. [2] characterize algebraic conditions under which Datalog over pre-semirings converges in finitely many iterations and semi-naive evaluation applies. These works provide incremental fixpoint constructions or convergence characterizations for particular Datalog semantics. Our contribution is complementary: we formalize FPD as a finite-prefix runtime problem for incrementally transformed circuits and establish its general limitations and its exact detectability for useful circuit classes.

Feldera implementation of DBSP. As mentioned in §5 and §6, the Feldera implementation of DBSP [14] uses internal-state stability to detect convergence. Its runtime gives each operator a scope-indexed fixed-point check, intended to report whether the operator's output will remain stable under stable inputs, and declares a circuit stable when all of its operators report stability. Our theory gives this established implementation strategy a declarative account: IntFP and IntConv specify internal-state stability denotationally, StFP connects the declarative criterion to circuit state, and the StFP detector gives an algorithmic formalization.

The main novelty lies in the formal guarantees, especially the completeness result. We prove that IntConv is sound and that the StFP detector is sound and complete. The global IntConv detector combines the StFP detector with fixed-input and zero-output checks across all bracketed sub-circuits, yielding sound and complete IntConv detection. We also prove that IntConv is complete for a large class of useful circuits, including regular circuits and their incrementally optimized forms (§7). Thus,

for these circuits, the internal-state-stability criterion detects every convergence admitted by the external specification. These results are machine-checked in Lean 4. Moreover, our level-indexed predicates and detectors give a formal, compositional account of nested bracketed circuits, whereas the Feldera SQL-to-DBSP compiler [14] rejects a nested recursive operator within recursive code. Therefore, our theory may help guide future extended implementations.

Differential Dataflow and its mathematical foundation. Differential Dataflow (DD) [26] also incrementalizes recursive queries over versions $T \times \mathbb{N}$, the DD counterpart of the delta-of-deltas setting in our counterexample. At the collection level, Egress selects the first repeated collection, at $i^* = \min\{i : W_{(t,i)} = W_{(t,i-1)}\}$; its differential implementation instead operates on $\delta W_{(t,i)}$. In this setting, $\delta W_{(t,i)} = 0$ does not imply a repetition, since $W_{(t,i)} - W_{(t,i-1)} = \sum_{s \leq t} \delta W_{(s,i)}$. Thus, FirstZero cannot soundly determine i^* , and the paper defines differential Egress using i^* without giving a finite-prefix criterion for obtaining it from the differential trace. Similar to DBSP, this also turns out to be a gap between the theory and the implementation of DD: its prototype instead reports convergence when no outstanding differences remain in the dataflow, a global internal-quiescence check similar in spirit to IntConv.

A later developed mathematical foundation for DD by Abadi et al. [1], distinguishes fixed-iteration egress from first-repetition egress, but formalizes only the fixed-iteration policy; it notes that first-repetition egress would require partial streams. Consequently, its correctness theorem does not provide a compositional denotational account of data-dependent first-repetition termination for unbounded incremental iteration. Similarly, Flo [21], a semantic foundation for progressive stream processing, requires bounded inner streams, leaving termination of unbounded inner iteration outside its scope. Our theory instead gives a sound criterion for such unbounded computations and identifies useful circuit classes for which that criterion is complete.

10 Conclusion

In this paper, we developed a formal theory of convergence detection to address the fundamental gap between the mathematical specifications and practical implementations of Fixpoint Detection (FPD) in modern incremental recursive computation frameworks like DBSP. We identified and formalized the FPD problem, demonstrated that the commonly suggested “FirstZero” heuristic is unsound for the “delta-of-deltas” incremental optimization for recursive computation, and proved that exact FPD is impossible for arbitrary expressive circuits. Within our theory, we defined internal convergence (IntConv) as a declarative criterion corresponding to the internal-state-stability strategy used by practical implementations, and proved that it is a sound sufficient condition for convergence. We also defined the State Fixpoint (StFP) predicate and a sound and complete StFP detector, providing semantic and algorithmic formalizations of this strategy. The sound and complete global IntConv detector combines the StFP detector with fixed-input and zero-output checks across all bracketed sub-circuits. These predicates and detectors treat nested bracketed circuits compositionally. Furthermore, we proved that IntConv is complete for a broad class of useful circuits, including arbitrarily nested while loops, Datalog queries, and their incrementally optimized versions. These results provide formal semantic guarantees for an existing implementation strategy, and our results have been formalized in Lean 4.

Our results are mainly theoretical, rather than an empirical performance evaluation. A natural direction for future work is to implement and compare the global IntConv detector based on the general StFP detector with specialized global detectors within a complete system. Such an evaluation should measure detector overhead and compare end-to-end incremental execution with non-incremental recomputation, since that comparison also depends on the query, workload, data representation, and other system-level optimizations.

Data-Availability Statement

This paper has an accompanying Lean 4 formalization of its results, including the impossibility result Theorem 4.7, which we have submitted as anonymous supplementary material and plan to submit for Artifact Evaluation. The development is described in §8.

References

- [1] Martín Abadi, Frank McSherry, and Gordon D. Plotkin. 2015. Foundations of Differential Dataflow. In *Foundations of Software Science and Computation Structures*. Springer, London, UK, 71–83. doi:10.1007/978-3-662-46678-0_5
- [2] Mahmoud Abo Khamis, Hung Q. Ngo, Reinhard Pichler, Dan Suciu, and Yisu Remy Wang. 2024. Convergence of Datalog over (Pre-) Semirings. *J. ACM* 71, 2 (2024), 8:1–8:55. doi:10.1145/3643027
- [3] Mario Alvarez-Picallo, Alex Eysers-Taylor, Michael Peyton Jones, and C.-H. Luke Ong. 2019. Fixing Incremental Computation: Derivatives of Fixpoints, and the Recursive Semantics of Datalog. In *European Symposium on Programming (Lecture Notes in Computer Science, Vol. 11423)*. Springer, Prague, Czech Republic, 525–552. doi:10.1007/978-3-030-17184-1_19
- [4] Giorgio Ausiello, Umberto Nanni, and Giuseppe F Italiano. 1990. Dynamic maintenance of directed hypergraphs. *Theoretical Computer Science* 72, 2-3 (1990), 97–117.
- [5] Jon Louis Bentley and James B Saxe. 1980. Decomposable searching problems I. Static-to-dynamic transformation. *Journal of Algorithms* 1, 4 (1980), 301–358.
- [6] Mihai Budiu, Leonid Ryzhyk, Gerd Zellweger, Ben Pfaff, Lalith Suresh, Simon Kassing, Abhinav Gyawali, Matei Budiu, Tej Chajed, Frank McSherry, and Val Tannen. 2025. DBSP: automatic incremental view maintenance for rich query languages. *The VLDB Journal* 34, 4 (2025), 39. doi:10.1007/s00778-025-00922-y
- [7] Mario Carneiro. 2019. Formalizing Computability Theory via Partial Recursive Functions. In *10th International Conference on Interactive Theorem Proving (ITP 2019) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 141)*, John Harrison, John O’Leary, and Andrew Tolmach (Eds.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 12:1–12:17. doi:10.4230/LIPIcs.ITP.2019.12
- [8] Tej Chajed. 2022. DBSP Lean 3 Formalization. <https://github.com/tchajed/database-stream-processing-theory>.
- [9] Pehong Chen, Michael A Harrison, and Ikuo Minakata. 1988. Incremental document formatting. In *Proceedings of the ACM conference on Document processing systems*. ACM, Santa Fe, NM, USA, 93–100.
- [10] Rada Chirkova and Jun Yang. 2012. Materialized views. *Foundations and Trends in Databases* 4, 4 (2012), 295–405. doi:10.1561/19000000020
- [11] Leonardo Mendonça de Moura and Sebastian Ullrich. 2021. The Lean 4 Theorem Prover and Programming Language. In *28th International Conference on Automated Deduction (Lecture Notes in Computer Science, Vol. 12699)*. Springer, Virtual Event, 625–635. <https://api.semanticscholar.org/CorpusID:235800962>
- [12] Jon Doyle. 1979. A truth maintenance system. *Artificial intelligence* 12, 3 (1979), 231–272.
- [13] Christian Fecht and Helmut Seidl. 1998. Propagating differences: An efficient new fixpoint algorithm for distributive constraint systems. In *European Symposium on Programming*. Springer, Lisbon, Portugal, 90–104.
- [14] Feldera Inc. 2025. Feldera repository. <https://github.com/feldera/feldera/tree/ecac53677479a3bfd12cd9551818fcd8fa246bc>. Commit ecac53677479a3bfd12cd9551818fcd8fa246bc (September 23, 2025).
- [15] John Field and Tim Teitelbaum. 1990. Incremental reduction in the lambda calculus. In *Proceedings of the 1990 ACM Conference on LISP and Functional Programming*. ACM, Nice, France, 307–322.
- [16] Bjorn N Freeman-Benson, John Maloney, and Alan Borning. 1990. An incremental constraint solver. *Commun. ACM* 33, 1 (1990), 54–63.
- [17] Carlo Ghezzi and Dino Mandrioli. 1979. Incremental parsing. *ACM Transactions on Programming Languages and Systems (TOPLAS)* 1, 1 (1979), 58–70.
- [18] Sergio Greco and Cristian Molinaro. 2015. *Datalog and Logic Databases*. Springer, Cham, Switzerland. doi:10.1007/978-3-031-01854-1
- [19] Todd J. Green, Zachary G. Ives, and Val Tannen. 2011. Reconcilable Differences. *Theory of Computing Systems* 49, 2 (2011), 460–488. doi:10.1007/s00224-011-9323-x
- [20] Ashish Gupta and Inderpal Singh Mumick (Eds.). 1999. *Materialized Views: Techniques, Implementations, and Applications*. The MIT Press, Cambridge, MA, USA.
- [21] Shadaj Laddad, Alvin Cheung, Joseph M. Hellerstein, and Mae Milano. 2025. Flo: A Semantic Foundation for Progressive Stream Processing. *Proceedings of the ACM on Programming Languages* 9, POPL (2025), 241–270. doi:10.1145/3704845
- [22] J van Leeuwen and Derick Wood. 1980. Dynamization of decomposable searching problems. *Inform. Process. Lett.* 10, 2 (1980), 51–56.

[23] Yanhong A. Liu. 2024. Incremental Computation: What Is the Essence? (Invited Contribution). In *Proceedings of the 2024 ACM SIGPLAN International Workshop on Partial Evaluation and Program Manipulation, PEPM 2024, London, UK, 16 January 2024*, Gabriele Keller and Meng Wang (Eds.). ACM, London, UK, 39–52. doi:10.1145/3635800.3637447

[24] L.A. Lombardi and B. Raphael. 1964. LISP as the Language for an Incremental Computer. In *The Programming Language LISP: Its Operation and Applications*, Edmund C. Berkeley and Daniel G. Bobrow (Eds.). The MIT Press, Cambridge, MA, USA, 204–219.

[25] Lionello A. Lombardi. 1967. Incremental Computation: The Preliminary Design of a Programming System Which Allows for Incremental Data Assimilation in Open-Ended Man-Computer Information Systems. In *Advances in Computers*, Franz L. Alt and Morris Rubinfeld (Eds.). Vol. 8. Academic Press, New York, NY, USA, 247–333. doi:10.1016/S0065-2458(08)60698-1

[26] Frank McSherry, Derek Gordon Murray, Rebecca Isaacs, and Michael Isard. 2013. Differential Dataflow. In *6th Biennial Conference on Innovative Data Systems Research. CIDRDB*, Asilomar, CA, USA, 12 pages. https://www.cidrdb.org/cidr2013/Papers/CIDR13_Paper111.pdf

[27] Robert Paige and Shaye Koenig. 1982. Finite differencing of computable expressions. *ACM Transactions on Programming Languages and Systems (TOPLAS)* 4, 3 (1982), 402–454.

[28] Ganesan Ramalingam and Thomas Reps. 1996. An incremental algorithm for a generalization of the shortest-path problem. *Journal of Algorithms* 21, 2 (1996), 267–305.

[29] G. Ramalingam and Thomas W. Reps. 1993. A Categorized Bibliography on Incremental Computation. In *Conference Record of the Twentieth Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, Charleston, South Carolina, USA, January 1993*, Mary S. Van Deusen and Bernard Lang (Eds.). ACM Press, Charleston, SC, USA, 502–510. doi:10.1145/158511.158710

[30] Thomas Reps, Tim Teitelbaum, and Alan Demers. 1983. Incremental context-dependent analysis for language-based editors. *ACM Transactions on Programming Languages and Systems (TOPLAS)* 5, 3 (1983), 449–477.

[31] Barbara G Ryder and Marvin C Paull. 1988. Incremental data-flow analysis algorithms. *ACM Transactions on Programming Languages and Systems (TOPLAS)* 10, 1 (1988), 1–50.

Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009